



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Internet Data Center - Georeplication

- part 2

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria
michele.colajanni@unibo.it

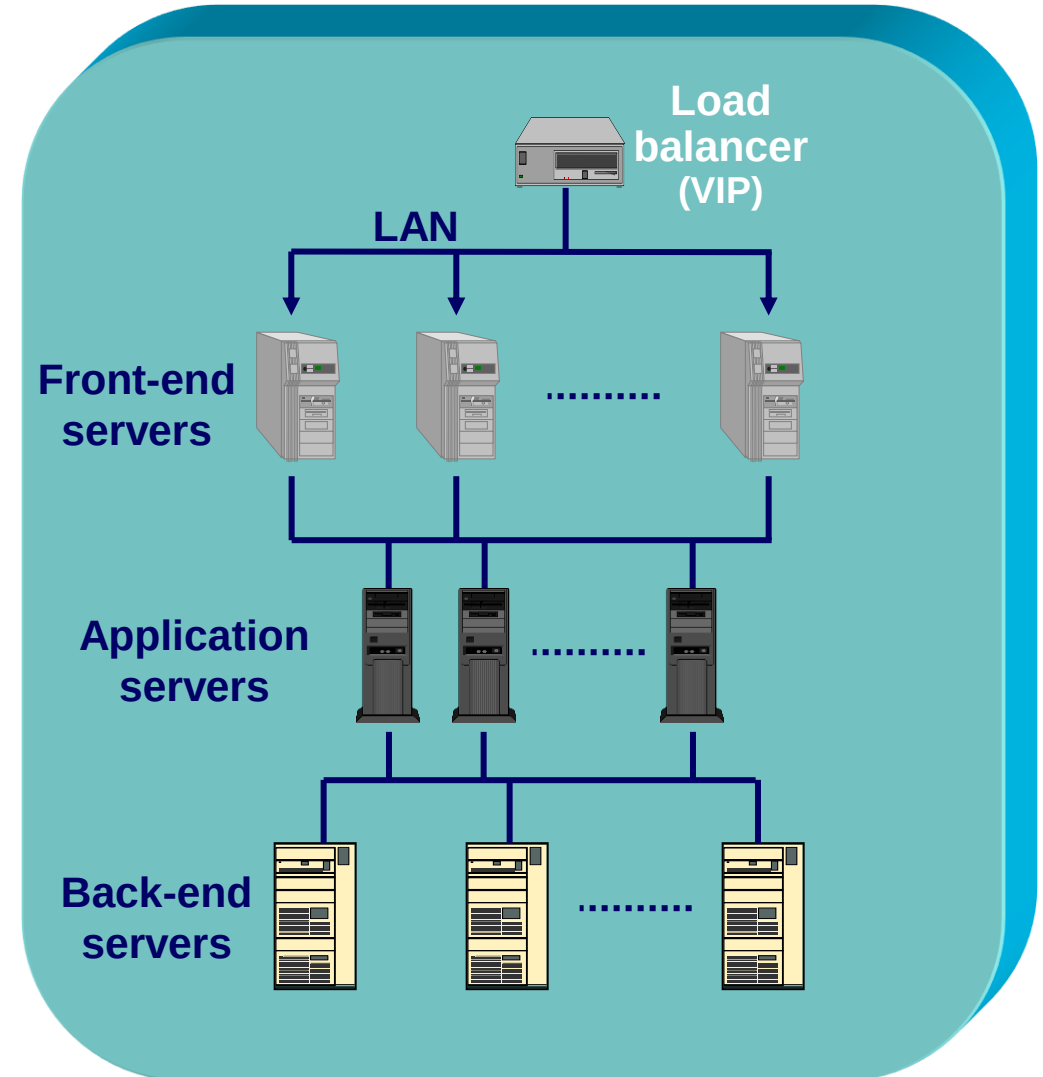
Geographical replication



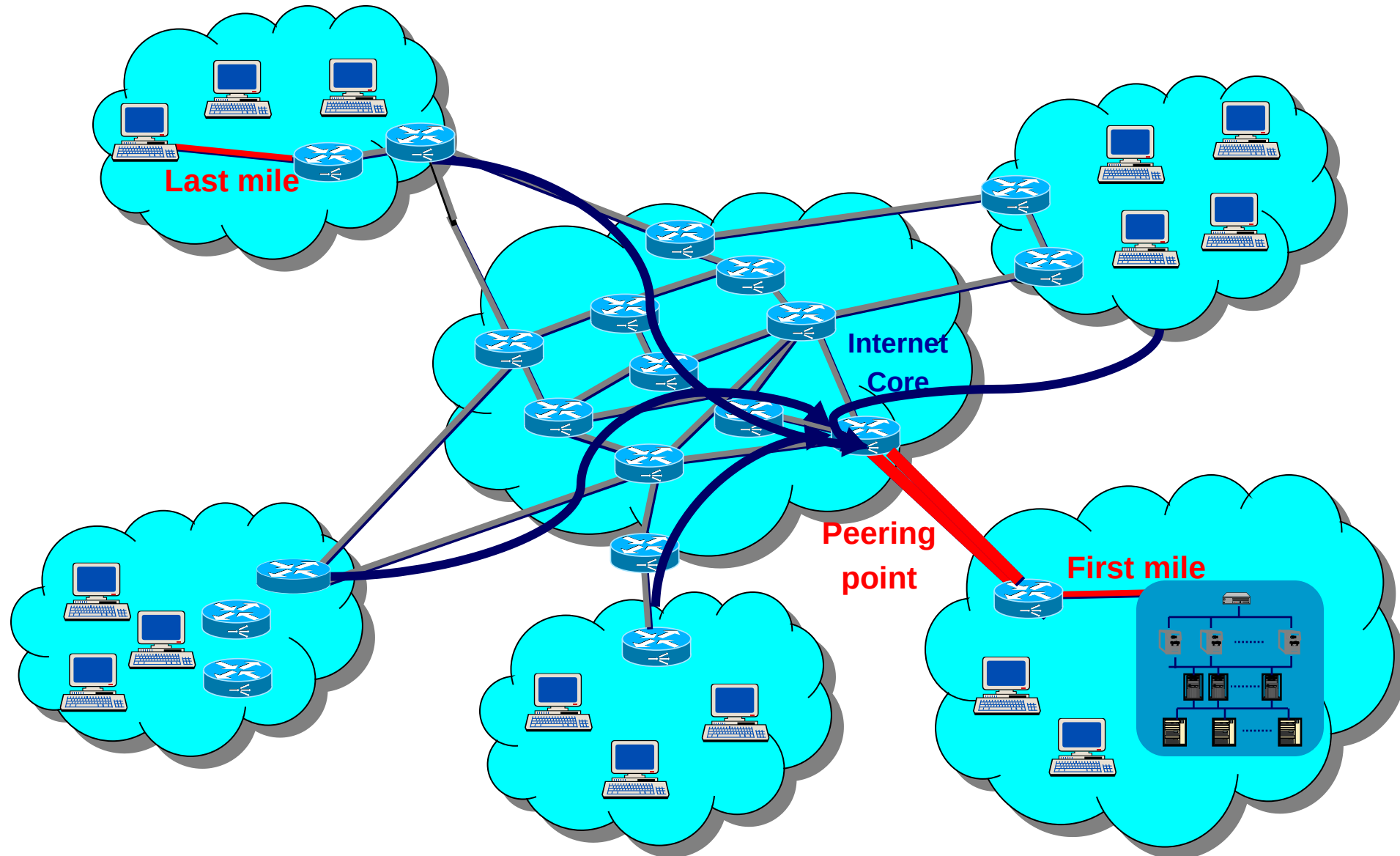
Let us take up again the previous question

1) How many servers can we add?

2) How many servers is it convenient to add?



One datacenter has evident limits of *reliability* and *scalability*



Reliability

If your datacenter is really popular, then you have high risks. But you can afford large investments, hence

- You cannot take any kind of risks about **internal reliability**
 - Infrastructure (remember: from T1 to T4 tier)
 - Employees (faithful, competent, available even at 2 AM?)
- You cannot take any kind of risks about **external reliability**
 - Network: several single points of failure and bottlenecks
 - Electrical power
 - Road works
 - Bad weather, floods, earthquakes, fire, ...
 - Human: cyber-attackers, political instability, strikes, health, ...



The drawbacks of the *external* network

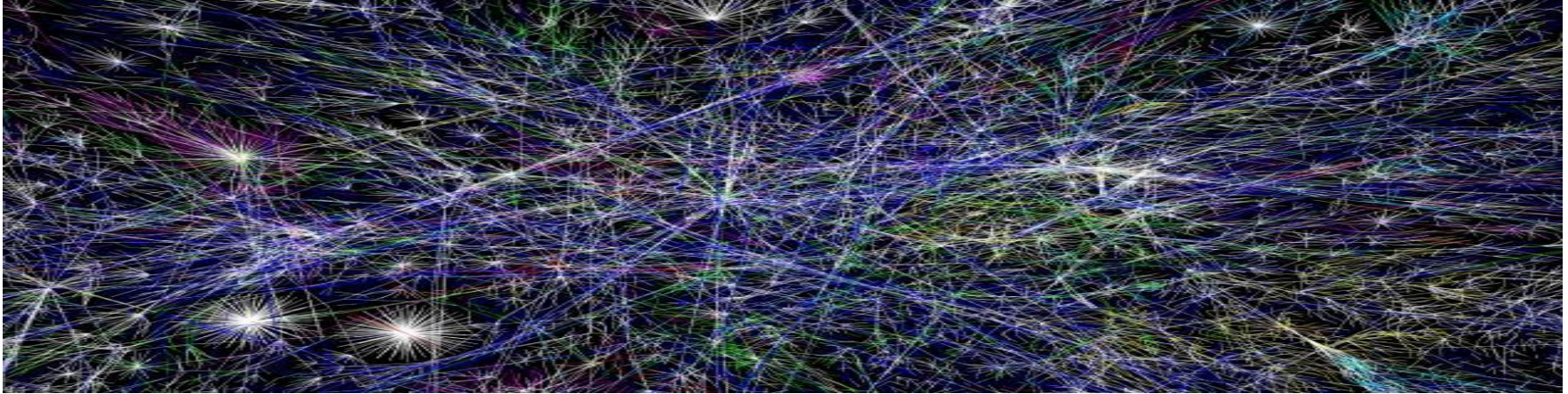
Modern services are provided by geographically distributed datacenters, but (typical) providers have no control over the external network of Internet Service providers (ISPs)

➔ Building *global-scale* services is tough

- no bandwidth guarantee because the network is a **black box** for most providers (not for Google!)
 - Good for a multi-owned heterogeneous Internet
 - Serious issue for a service provider owning one data center:
 - using inefficient overlay networks, because of the mismatch between physical topology and logical routing
 - trying to infer network properties that typically change dynamically



Internet Service Provider



Internet Service Providers

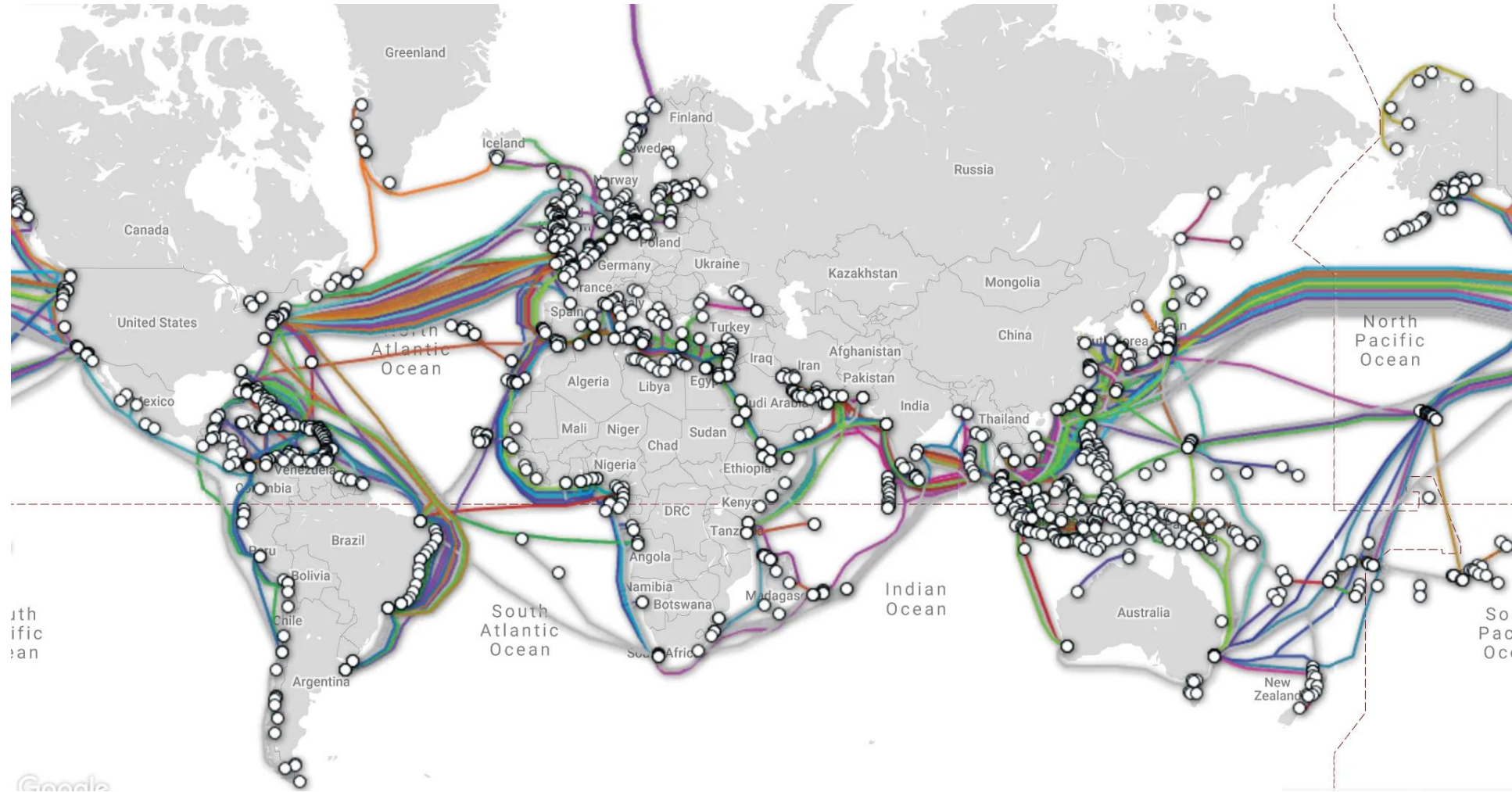
(interconnections – fiber, cable, wireless, satellite)

Examples:

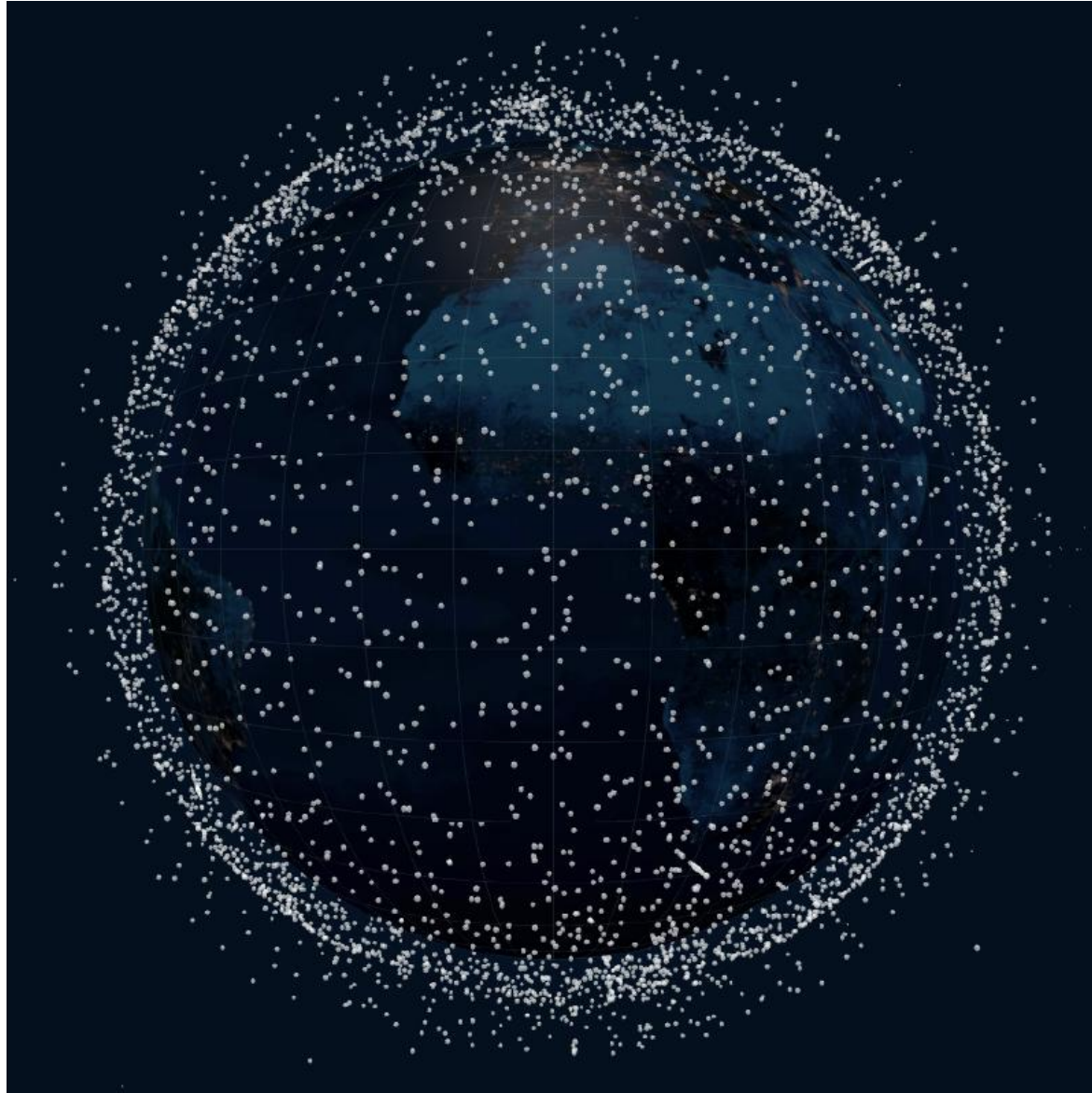
AT&T, Verizon, Vodafone, Comcast, COX, Viasat, Lumen, China Telecom, Tata Communications, Tencent, Jio, ...



Undersea connections (99% of traffic)



Satellite connections (1% - *growing trend*)



Another creature of Elon Musk



Not just cables, antennas and satellites ...

How is Internet organized?

Anarchy or some organization?

Centralized organization or Distributed organization?

Anything in common?

YES

Organization: *Autonomous Systems*

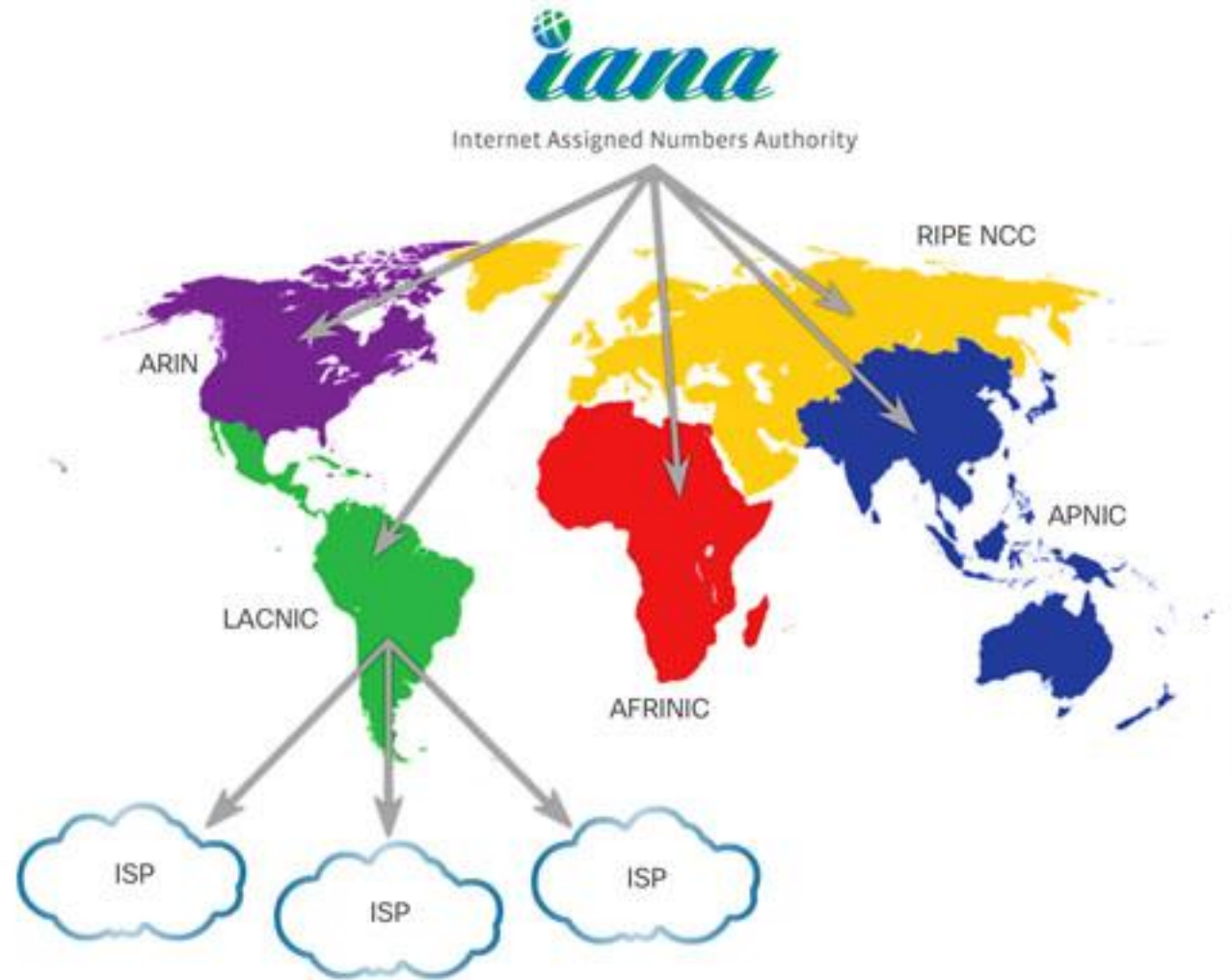
Identity: *Internet address (IP)*

Communication protocols: *TCP/IP*

Another identity and resolution method: *DNS*

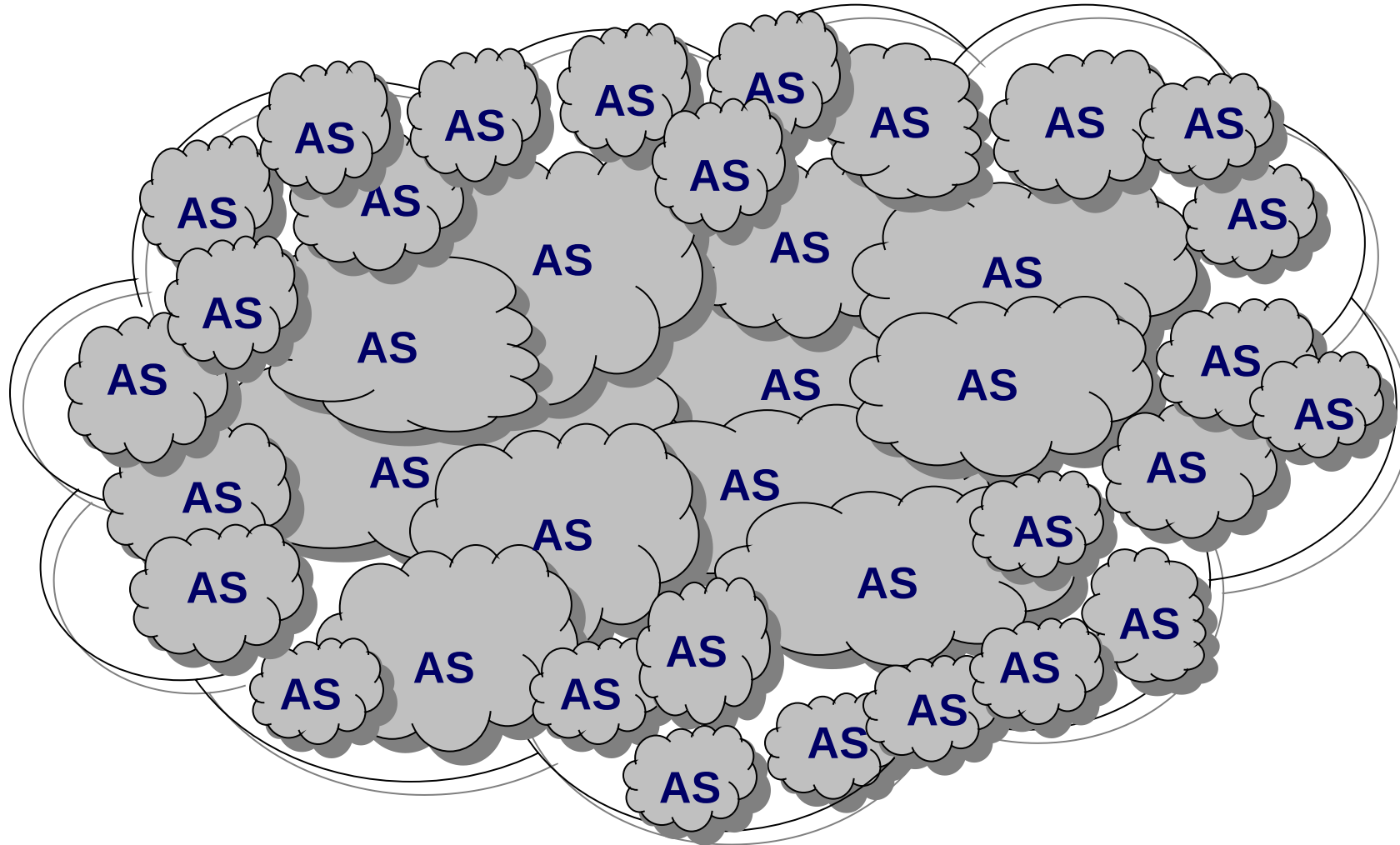


IANA: Distribution of IP addresses



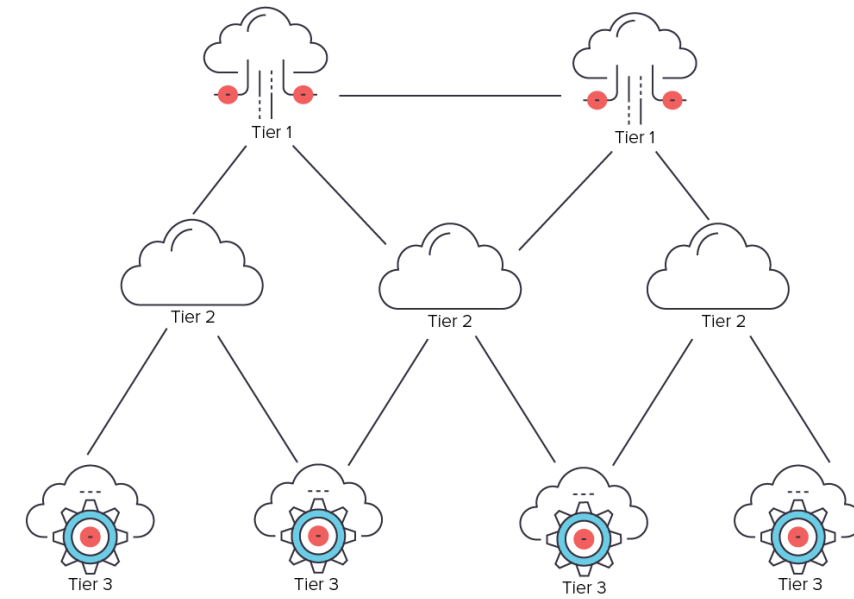
Internet is organized in *Autonomous Systems*

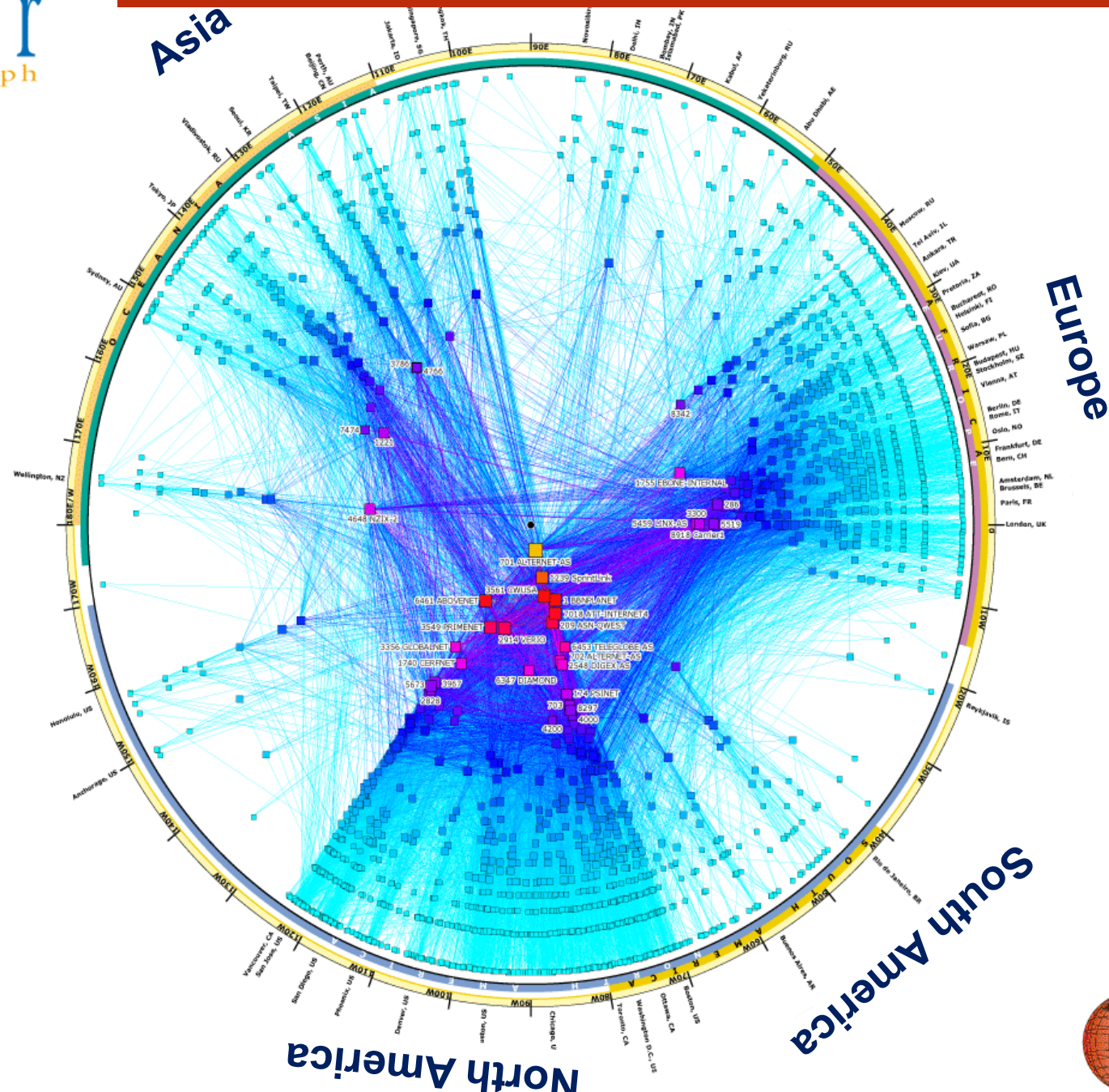
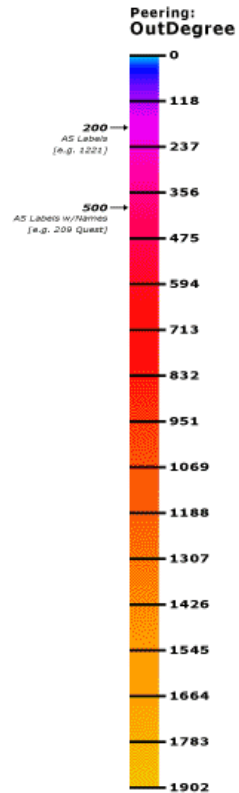
(Each AS is owned and managed by one ISP)



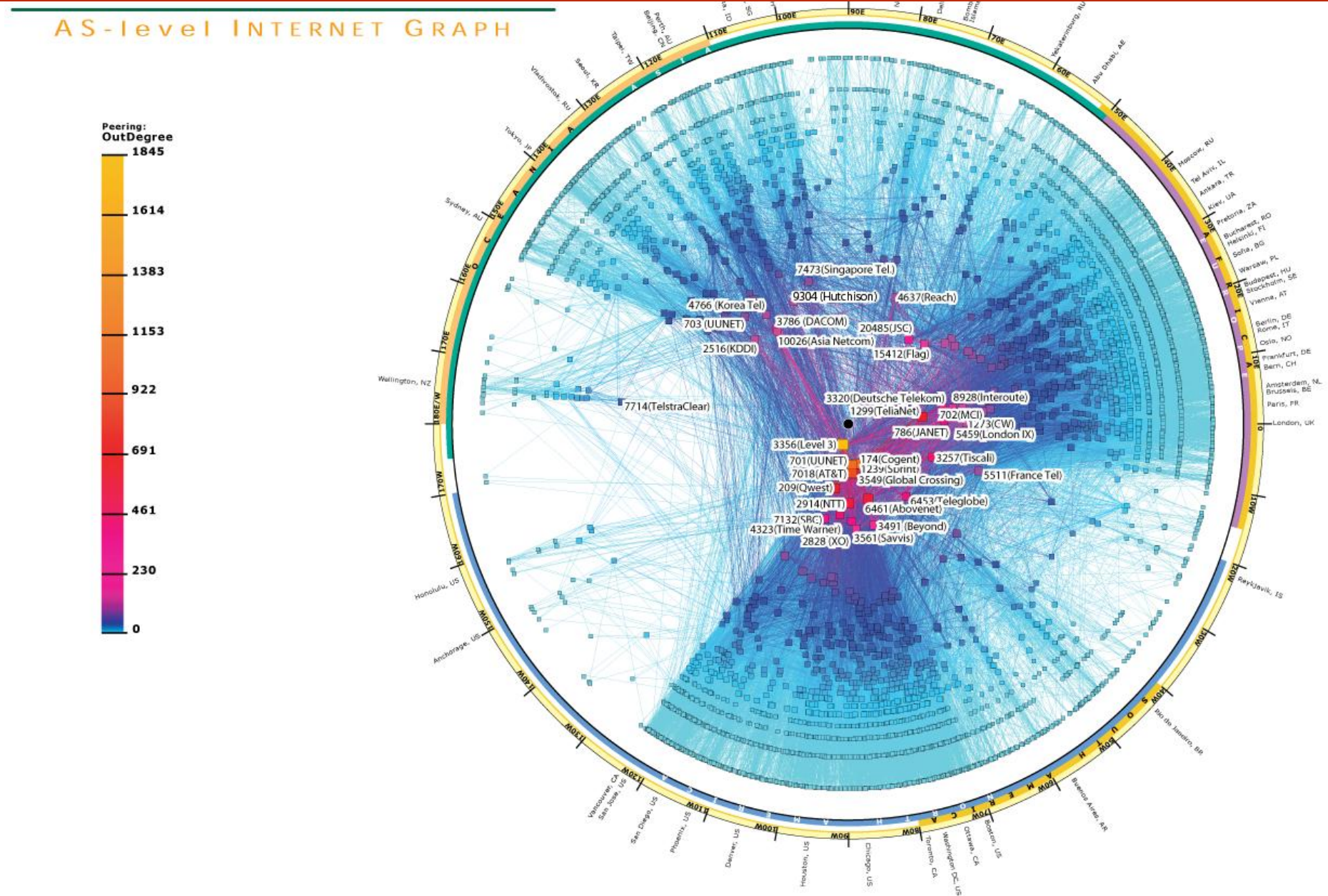
Tier 1 ISP

- 1) AT&T Services (AS7018), US
- 2) Century Link (AS3356), US
- 3) Deutsche Telekom (AS3320), Germany
- 4) GTT Communications (AS3257), US
- 5) Liberty Global (AS6830), US-UK-NL
- 6) NTT Communications (AS2914), Japan
- 7) Orange (AS5511), France
- 8) PCCW Global (A3491), Hong Kong
- 9) Sprint Nextel Corporation (AS1239), US
- 10) TATA Communications (AS6453), India
- 11) Telecom Italia Sparkle (AS6762), Italy
- 12) Telxius (AS12956), Spain
- 13) Telia Carrier (AS1299), Sweden
- 14) Verizon Business (AS701), US
- 15) Zayo Group (AS6461), US

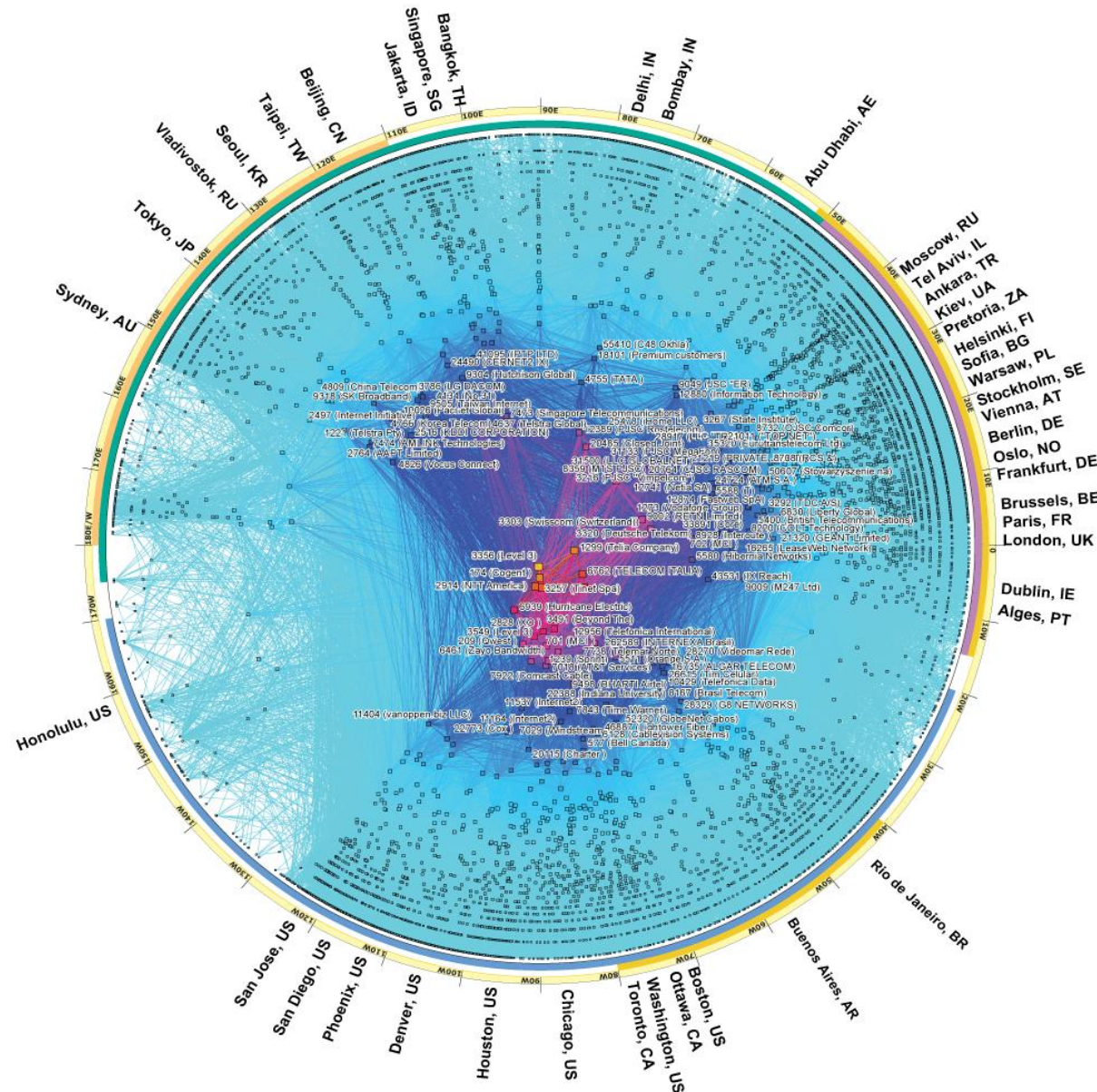




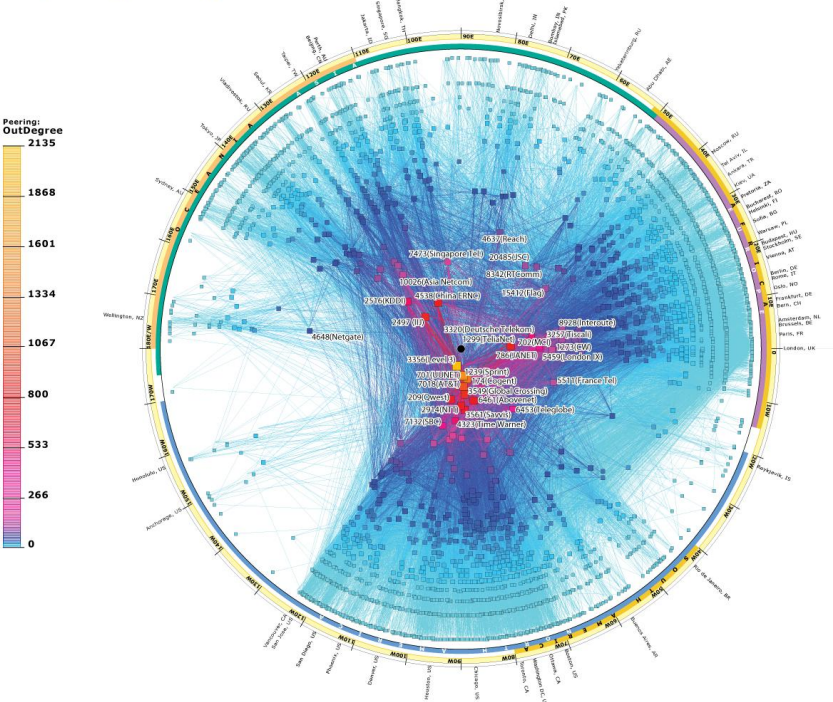
Interconnections among AS (2008)



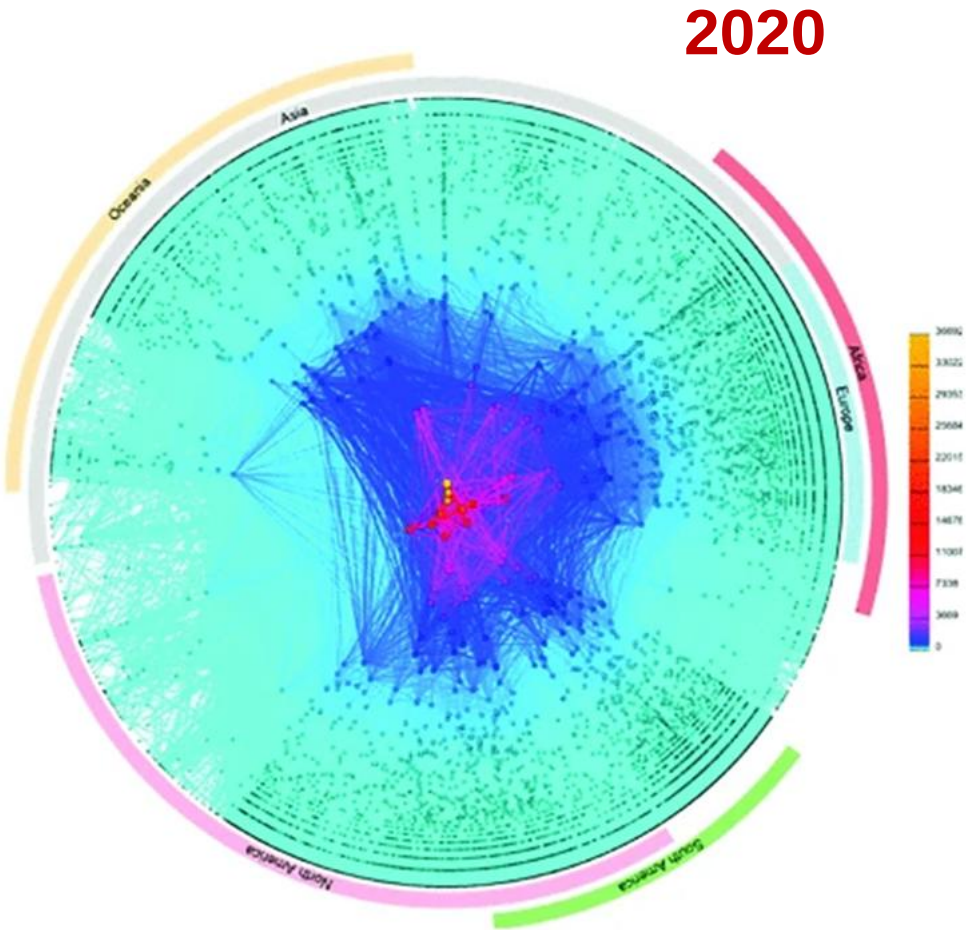
Interconnections among AS (2017)



Interconnections among Autonomous Systems



2007



2020



Geographical replication

If network latency and availability matter → there is a strong motivation for geographically distributing services around the world:

1. **Improving availability** through *redundancy*, as not all areas are likely to lose power, experience an earthquake or an hurricane, or suffer strikes and social riots at the same time
2. **Decreasing client-server latency due to speed-of-light “delays”**



If you are a cloud provider or you manage a popular service, one datacenter is insufficient

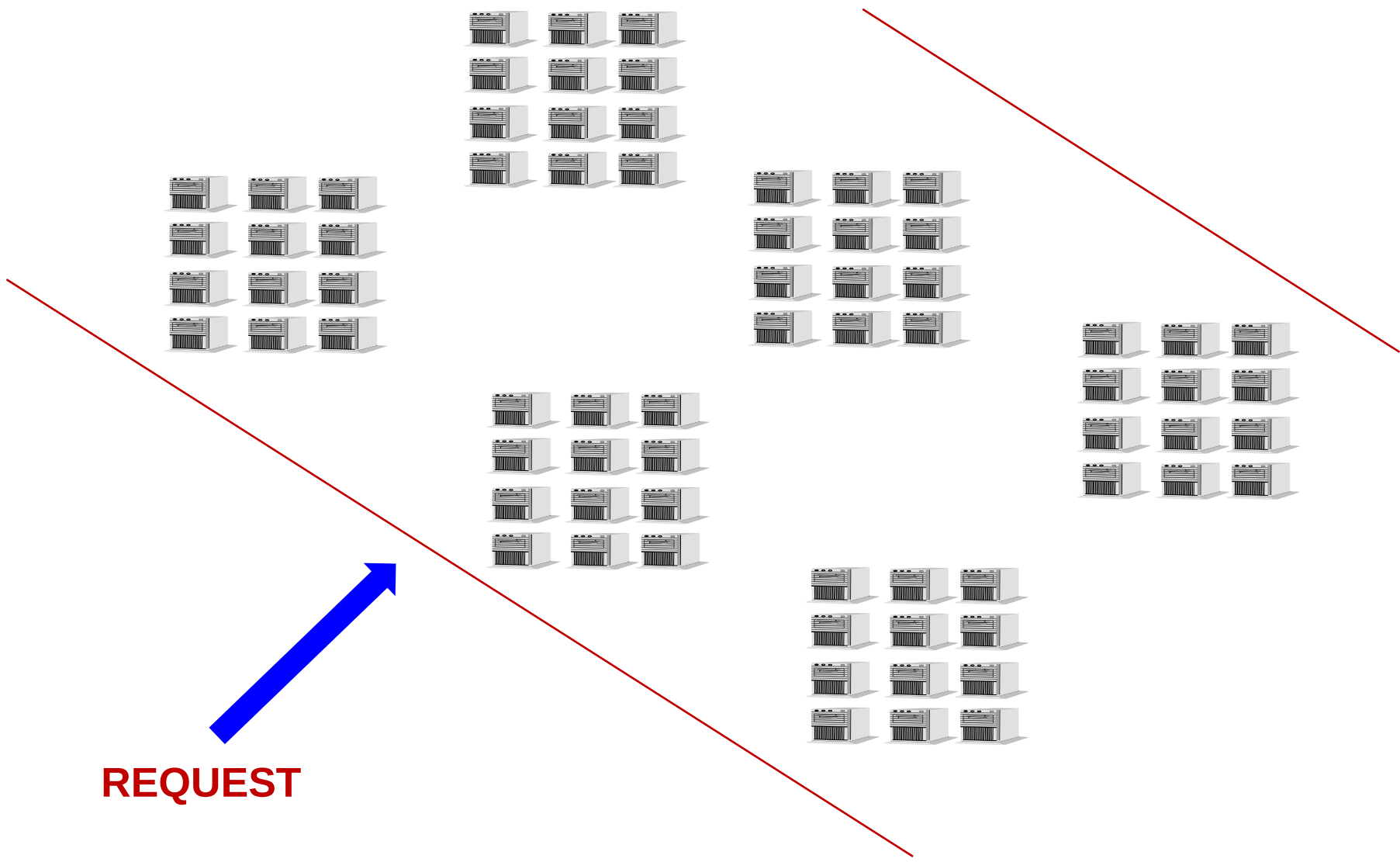
Multiple **Internet Data Centers (and not just one)** represent the back-end infrastructure used by cloud providers, largest digital companies and AI providers to yield agile, elastic, reliable, secure services delivered through the Internet



The dispatching problem



Resource selection



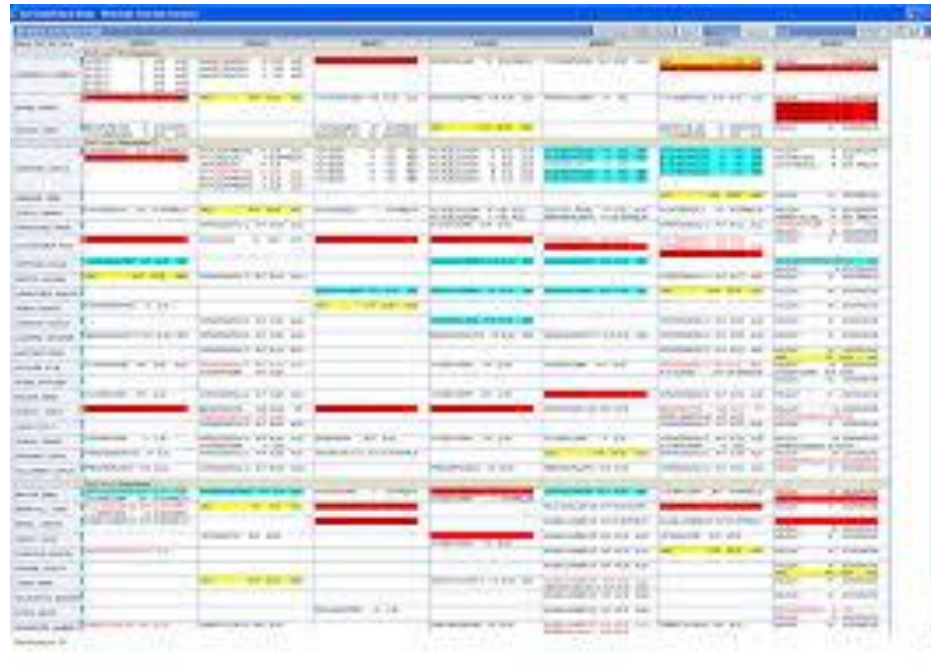
The theory comes from scheduling

- **Offline scheduling**
- **Online scheduling**
- Scheduling for **hard real-time systems**



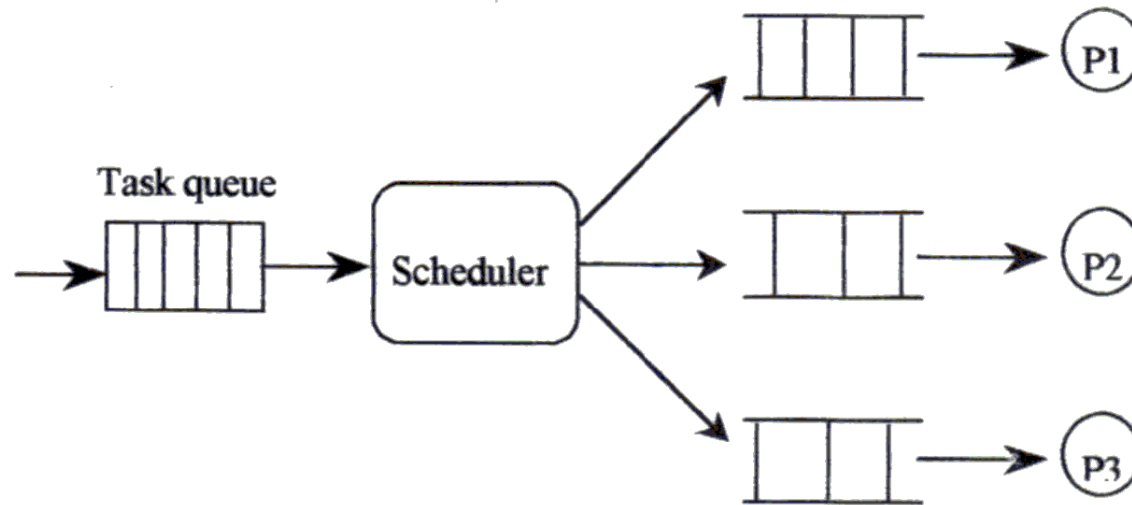
Offline scheduling

- In **offline scheduling** problems, there is no conception of “point of time”. Algorithms stay outside the real-time world and can see the past and future of tasks
- Hence, the algorithms know the total knowledge of the problem before we make decision → **optimization is a possible goal**
- Examples: timetable, crew assignment, guard duty, industrial production, Gantt chart, etc.



Online scheduling

- **Online scheduling** is characterized by making decision “online”, which means a point in the time axis. In this point of time, the algorithms cannot see the future jobs or tasks, whereas they only know the jobs or tasks before or at this point of time → ***just approximation of optimum is possible***
- **Temporal term of scheduling:** Long-term, Mid-term, Short-term



Online scheduling

An **online scheduling algorithm** decides which requests are given access to resources from moment to moment, e.g.,

- **Operating systems:** multiple processes competing for the CPU
- **Hypervisor:** virtual machines
- **Router:** packets competing for network links
- **RAID drive:** read/write operations competing for disks
- **Print manager:** prints



Possible goals of online scheduling

- Minimizing (average) response time
- Maximizing throughput
- Maximizing availability
- Fairness in sharing resources
- Meeting deadlines
- Avoiding starvation
- Load balancing
- Load sharing
- ...
- Combination of two or multiple goals



Online scheduling algorithms

1. ***First Come First Served*** (FCFS)
2. ***Round Robin*** (RR)
3. ***Round Robin with quantum***
4. ***Weighted Round Robin*** (WRR)
5. ***Shortest Job First*** (SJF), also known as ***Shortest Time to Completion First*** (STCF)
6. ***Shortest Remaining Time First*** (SRTF), also known as ***Shortest Remaining Time to Completion First*** (SRTCF), that is the preemptive version of SJF
7. ***Priority*** (with and without preemption)
8. ***Multi-level***: multiple queues, each with its scheduling algorithm
9. ***Multi-level with fairness***



Scheduling and Dispatching

- **Dispatching** is more related to *short-term online scheduling* algorithms, hence in our contexts we prefer to use *dispatching* instead of *scheduling*
- There are many definitions and interpretations that are applicable to different contexts. My favorite:
 - **Dispatching** is “what next and where”
 - **Scheduling** is “what, where, when, why”



Classification

- **Dispatching contexts**
 - Local area
 - Geographical area
- **Dispatching mechanisms - *local***
 - Switch (local)
 - Layer 4
 - Layer 7
- **Dispatching algorithms - *geo***
 - DNS resolution (geo)
 - HTTP redirection (geo)

Dispatching algorithms

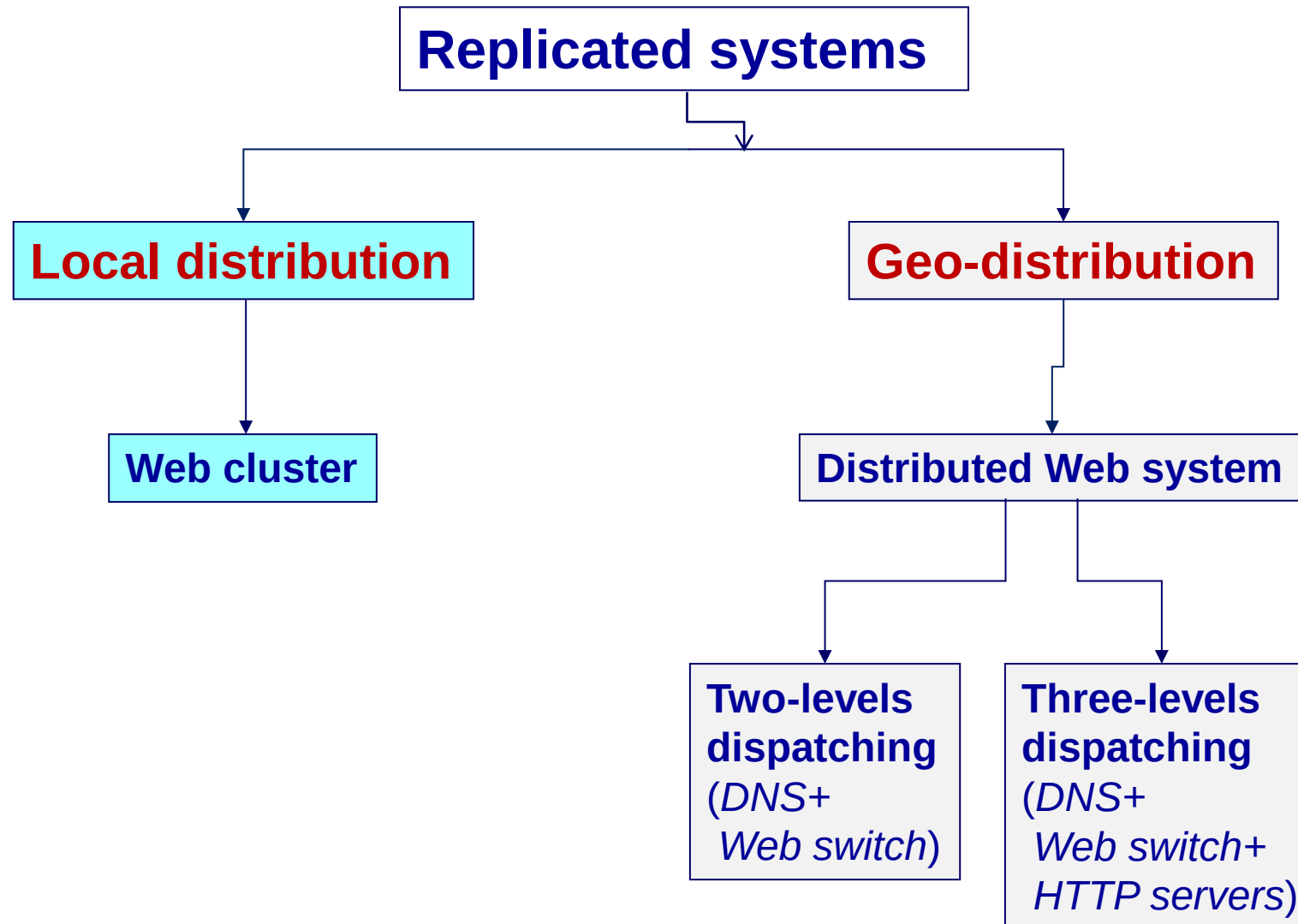
- Static
- Dynamic
- Adaptive

Dispatching algorithms

- Static
- Dynamic
- Adaptive



Taxonomy



Dispatching mechanisms

1. Switch dispatching
2. DNS dispatching
3. HTTP dispatching (*HTTP redirection*)

Context

Local

Global

Global



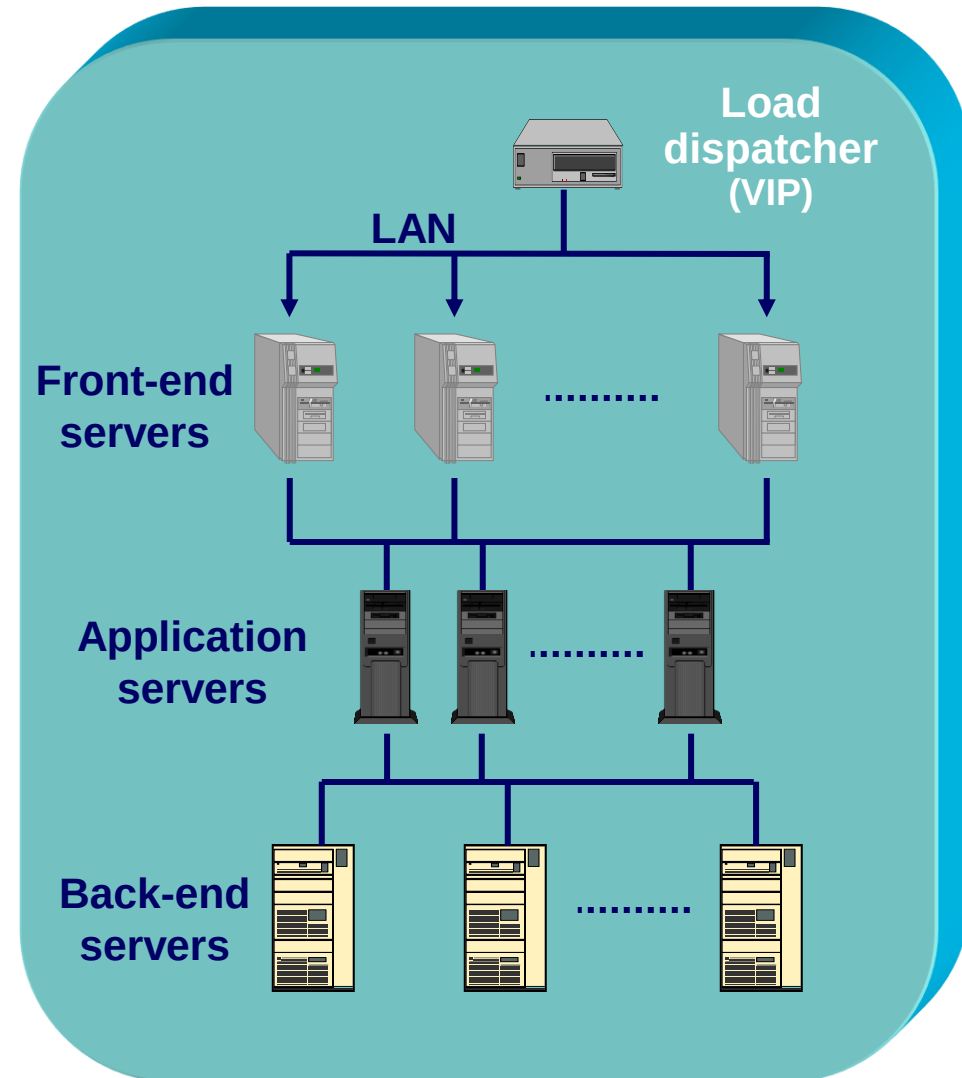
Local dispatching



Locally distributed system

A locally distributed architecture has

- One hostname (e.g., “www.unibo.it”)
- One IP address (*virtual IP address*) of the **front-end switch**
 - Only its IP address is visible and corresponds to the site IP address)
 - All internal servers have hidden IP addresses



1. Switch dispatching

The dispatcher of a Web cluster has total control on all requests reaching the cluster

It can use **client** information, **server** information or **client and server** information

- Client information can be ***content-blind*** (switch at layer 4)
- Client information can be ***content-aware*** (switch at layer 7)



Server info-aware algorithms

- Request assignment based on *server load info*
 - **Least loaded server** (← AVOID!)
 - **Random(k)+least loaded**: pick k servers randomly and choose the least loaded
 - **Weighted Round-Robin (WRR)**
 - it allows configuration of weights as a function of the server load
- Possible metrics to estimate the server load
 - **Input metrics**: information get by the Web switch without server cooperation, e.g., *Active connections*
 - **Server metrics**: information get by the servers and transmitted to the Web switch, e.g., *CPU/Disk utilization, response time*
 - **Forward metrics**: information get directly by the Web switch, e.g., *emulation of requests to servers*



Client info-aware algorithms

Session identifiers

- HTTP requests with same **SSL id** or same **cookie** assigned to the same server
 - Goal: avoid multiple client identifications for the same session

Content partition

- Content partitioned among servers according to **file type** (HTML, image, dynamic content, audio, video, ...)
 - Goal: use specialized servers for different contents
- Content partitioned among servers according to **file size** (Thresholds may be chosen dynamically)
 - Goal: augment load balancing
- File space partitioned among the servers through a hash function
 - Goal: improve *cache hit rate* in Web servers



Client info-aware algorithms

Content Aware Partition (CAP)

- Resource classification according to the impact of HTTP requests on main server components, e.g.,
 - *Low impact* (small-medium static files)
 - *Network bound* (large file download)
 - *Disk bound* (database queries)
 - *CPU bound* (“secure” requests)
- Cyclic assignment of each class of requests to servers
- Goal: to augment load sharing of *component bound* requests among servers

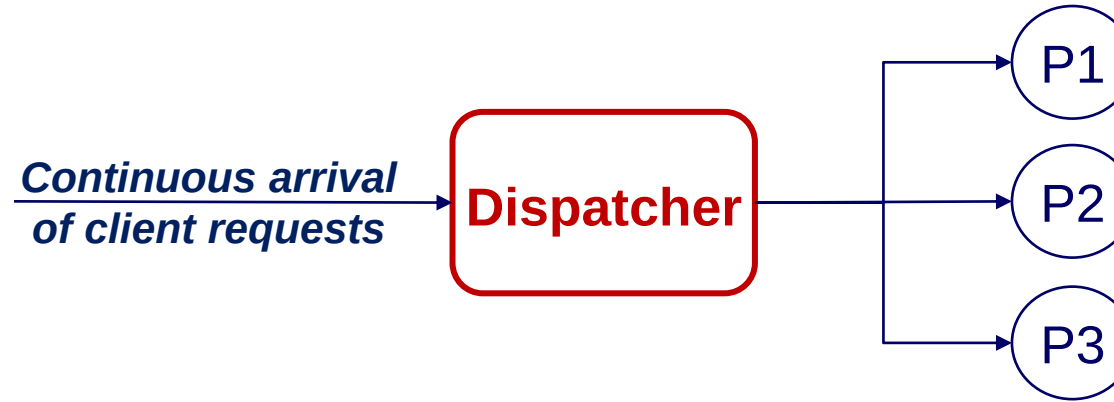


Dispatching algorithms

1. **Static** → *information-less*
2. **Dynamic** → *they get continuous information from the environment*
 - Client info-aware
 - Server info-aware
 - Client info-aware and Server info-aware
3. **Adaptive** → *they use different algorithms and apply the “best” one depending on system’s status*



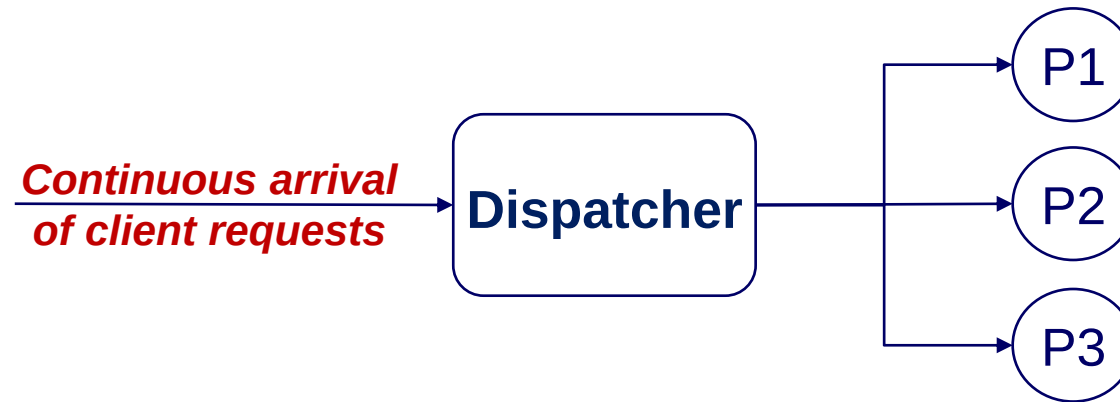
Static algorithms



- **Random**
 - no information about the system state
 - no history about previous assignments
- **Round Robin (RR)**
 - no information about the system state
 - history is about the previous assignment only



Dynamic: possible *client* information



1. IP address
2. Source port
3. Novel request (TCP syn)
4. HTTP vs HTTPS connection
5. Type of request (URL)
6. Cookie

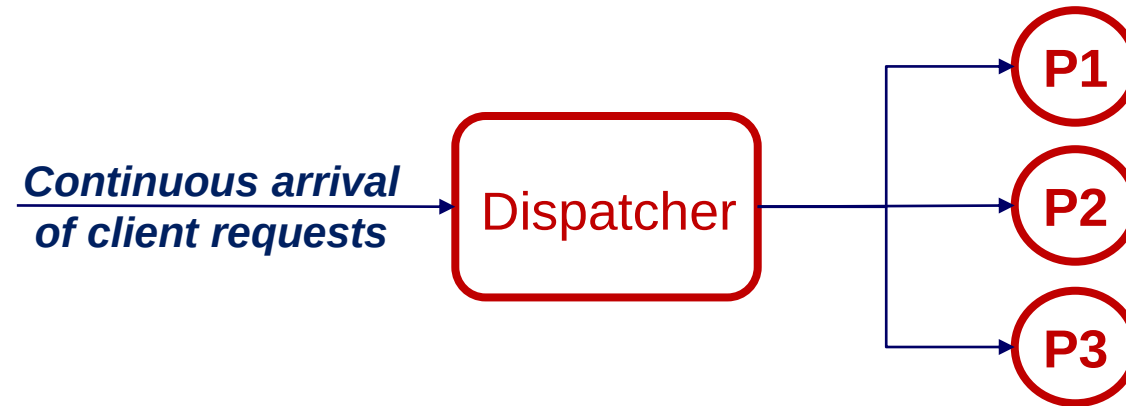
Client info-aware algorithms

Client partition

- Request assignment based on client information in inbound packets
 - Client IP address
 - Client port



Dynamic: possible *server* information



1. Information about servers known at the dispatcher level
 - Number of active requests
 - Number and type of active requests
2. Information provided by the servers
 - Length of process queue
 - Resource utilization (CPU, disk, network, memory)
3. Information measured by the dispatcher
 - Response time (measured through request probes)

Server info-aware algorithms

- Request assignment based on *server load info*
 - **Least loaded server** ($\leftarrow$ AVOID!)
 - **Random(k)+least loaded**: pick k servers randomly and choose the least loaded
 - **Weighted Round-Robin (WRR)**
 - it allows configuration of weights as a function of server load
- Possible metrics to estimate server load
 - **Input metrics**: information get by the Web switch without server cooperation, e.g., *Active connections*
 - **Server metrics**: information get by the servers and transmitted to the Web switch, e.g., *CPU/Disk utilization, response time*
 - **Forward metrics**: information get directly by the Web switch, e.g., *emulation of requests to servers*



Other combinations

- **Client info-aware + Server info-aware**
- **Adaptive**: dynamic algorithms that update their decisions based on significant changes in system's conditions



Client info-aware algorithms

- **Session identifiers**

- HTTP requests with same **SSL id** or same **cookie** assigned to the same server
 - Goal: avoid multiple client identifications for the same session

- **Content partition**

- Content partitioned among servers according to **file type** (HTML, image, dynamic content, audio, video, ...)
 - Goal: use specialized servers for different contents
- Content partitioned among servers according to **file size** (Thresholds may be chosen dynamically)
 - Goal: augment load balancing
- File space partitioned among the servers through a hash function
 - Goal: improve *cache hit rate* in Web servers



Client info-aware algorithms

- **Content Aware Partition (CAP)**

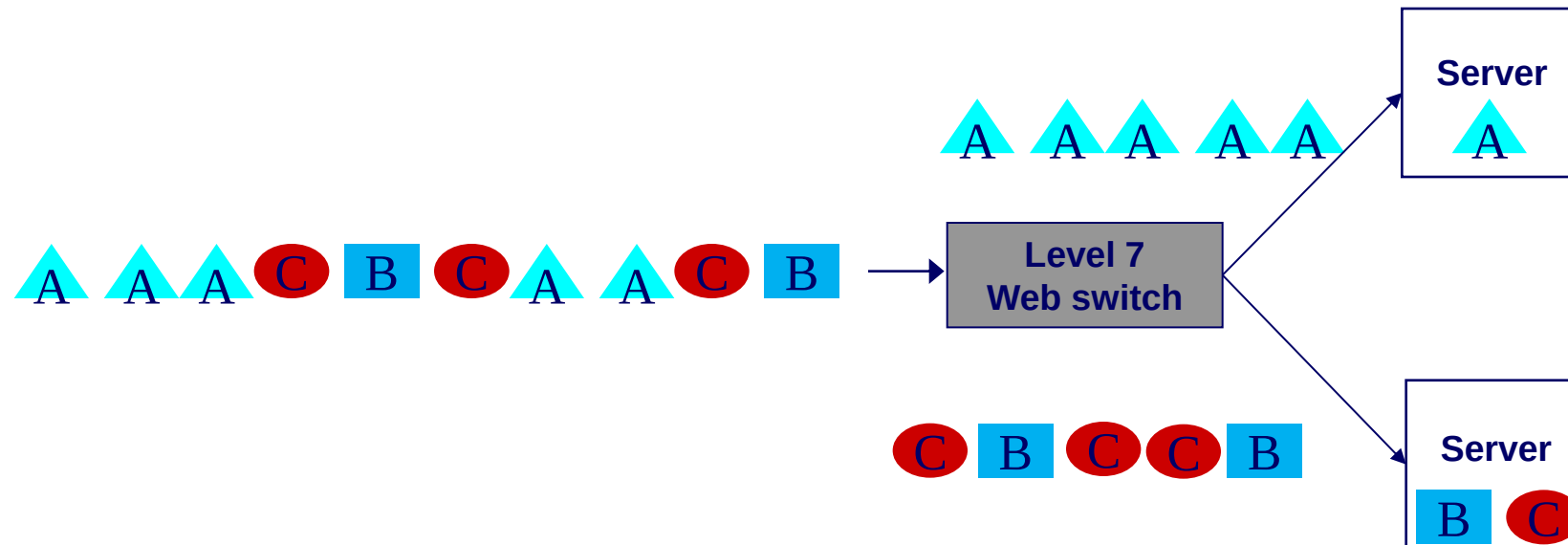
- Resource classification according to the impact of HTTP requests on main server components, e.g.,
 - *Low impact* (small-medium static files)
 - *Network bound* (large file download)
 - *Disk bound* (database queries)
 - *CPU bound* (“secure” requests)
- Cyclic assignment of each class of requests to servers
- Goal: to augment load sharing of *component bound* requests among servers



Client and server info-aware algorithm

Locality-Aware Request Distribution (LARD)

- First request for a given target assigned to the least loaded server (metrics: *number of active connections*)
- Subsequent requests for the same target assigned to the previously selected server
- Goal: improve locality (*cache hit rate*) in server cache



Geo dispatching



Geographically distributed system

- ***Web-based service realized on an architecture of geographically distributed Web clusters***
- **Web site addresses**
 - One hostname (e.g., “www.site.com”)
 - One IP address for each Web cluster

First level dispatching

The authoritative DNS, at the lookup, selects one Web cluster

Second level dispatching

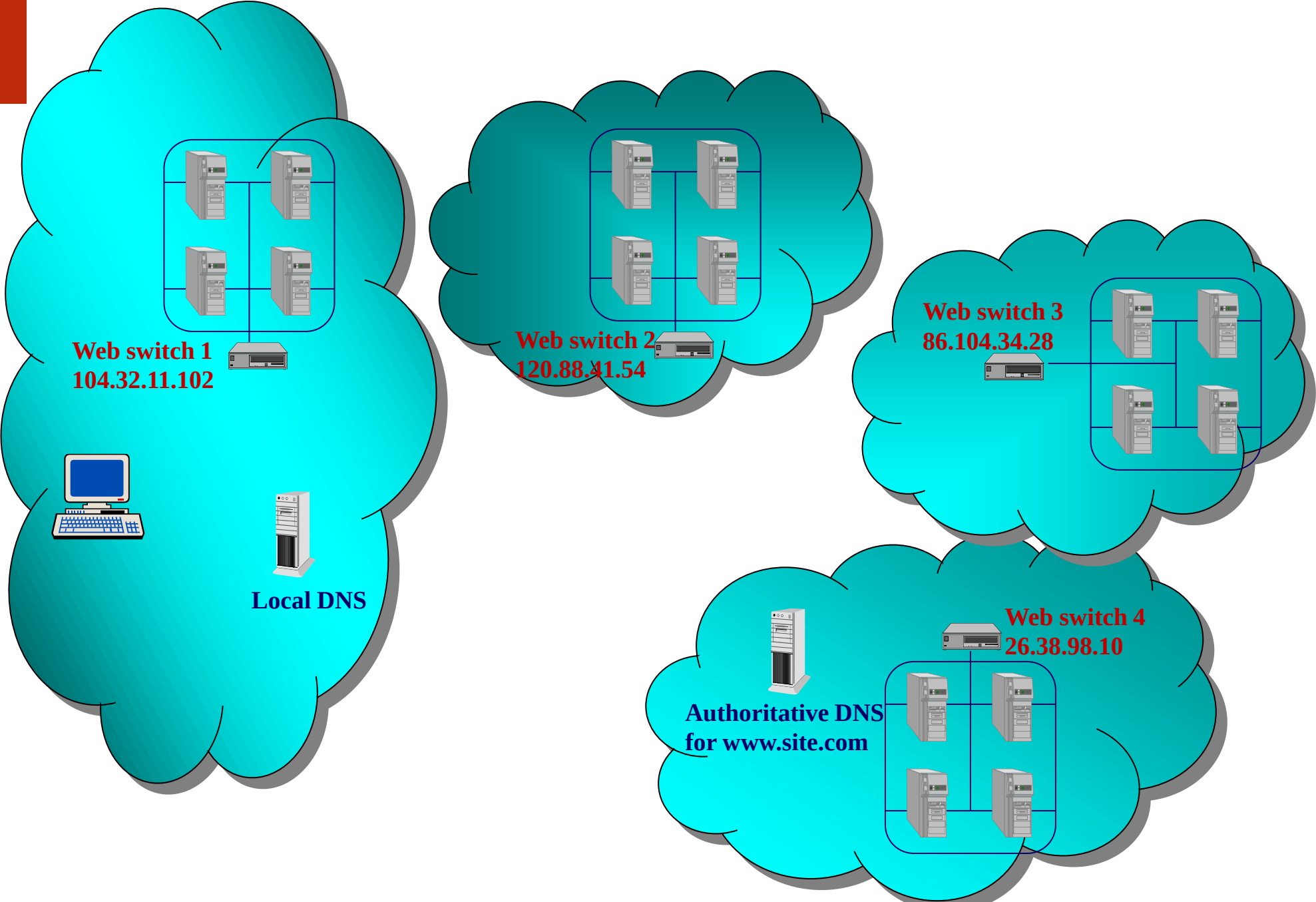
The Switch of the Web cluster selects one internal server

Third level dispatching

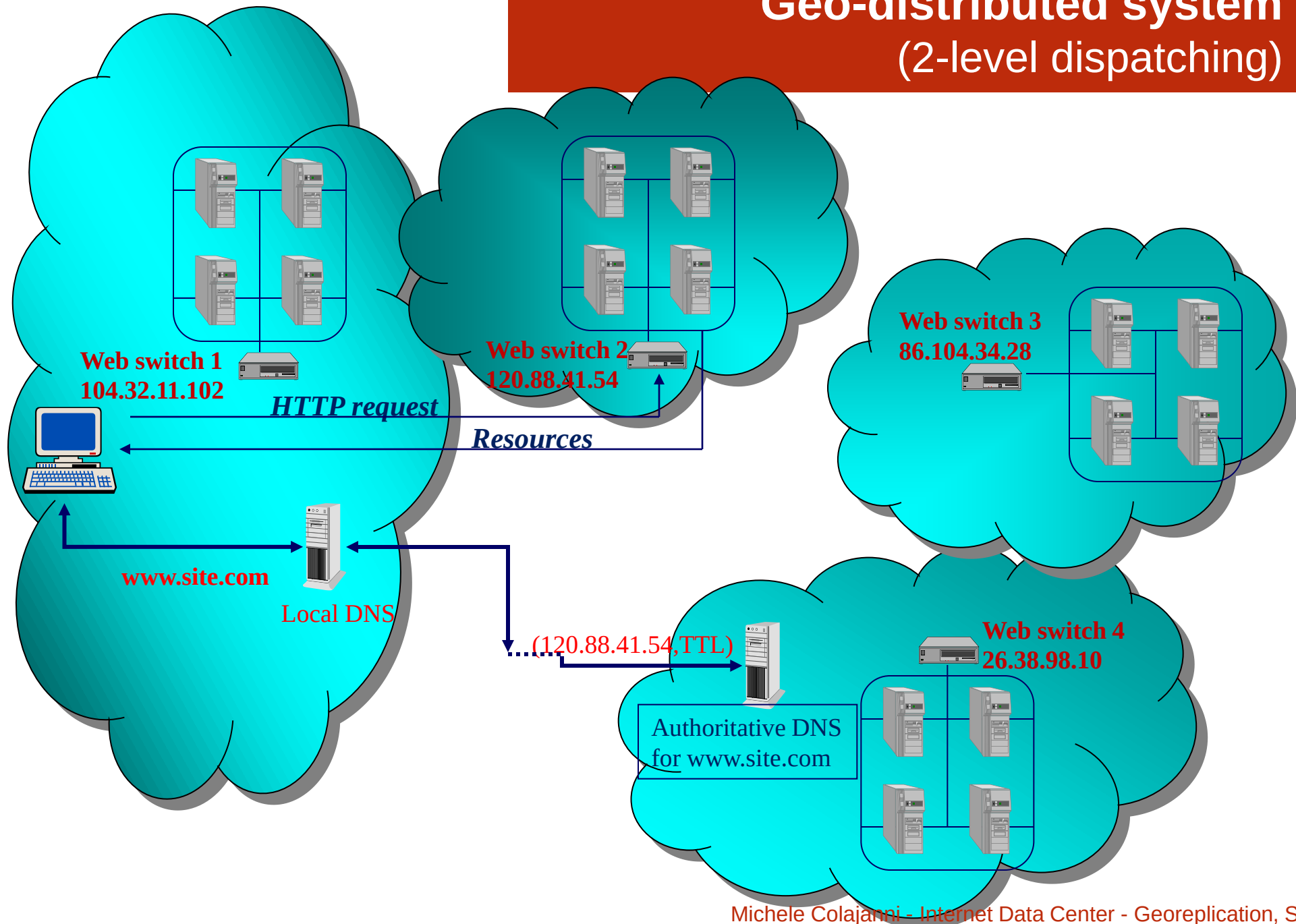
The Switch (layer 7) may redirect the received request to another Web cluster



Geo-distributed system



Geo-distributed system (2-level dispatching)



DNS dispatching

- The distributed Web cluster architectures implement global dispatching by intervening in the **lookup phase** of the address request:
 - A client asks for the **IP address** of a Web server corresponding to the **hostname** in the URL
 - If the hostname is valid, it receives the couple

(IP address, TimeToLive)

- The **enhanced authoritative DNS** of the Web site (or **another entity** that replaces or integrates the authoritative DNS) can use various dispatching policies to select the “best” Web cluster

NOTE: Errors or delays at DNS are often cause of service issues



Issues of DNS dispatching

Typical issues

- Load spikes in some hours/days

Additional issues

- Traffic depending on time zones
- Client distribution among Internet zones (AS)
- Proximity between the client and the Web cluster
- Caching of [hostname-IP] at intermediate DNS servers for TTL interval

All modern geographically distributed architectures use some sort of DNS dispatching



Issues of DNS dispatching

1. **[Hostname - IP address] caching problem:** the authoritative DNS of highly popular Web-based services controls only 5-7% of traffic reaching the servers of the site
2. **Internet proximity problem:** which is the cluster closest to the client? Is the cluster closer to the client always the best choice?

Unlike the switch (controlling 100% traffic), the DNS should use sophisticated dispatching algorithms



Caching issue: actions on TTL values

- **Constant TTL**

- Set TTL=0 to augment the DNS control
- Drawbacks
 - Non-cooperative name servers (they may ignore small TTL values <300 seconds)
 - Browser caches
 - Risk of overloading authoritative name servers

- **Adaptive TTL**

- Tailor TTL value adaptively for each address request by taking into account the popularity of client Internet domain and Web server loads



Internet proximity issue

- Internet proximity is an interesting issue

**Client-server *geographic proximity* does not mean
Internet proximity (round trip latency)**

- **Static information**

- client IP address to determine Internet zone (geographical distance)
- hop count (“stable” more than “static” information)
 - *network* hops (e.g., `traceroute`)
 - *Autonomous System* hops (routing table queries)

It does not guarantee selection of the best connected Cluster because “links are not created equal”



Internet proximity

- **Dynamic evaluation of proximity**
 - round trip time (e.g., `ping`, `tcping`)
 - available link bandwidth (e.g., `cprobe`)
 - latency time of HTTP requests (request emulation)

Additional time and traffic costs for evaluation

A related open issue:

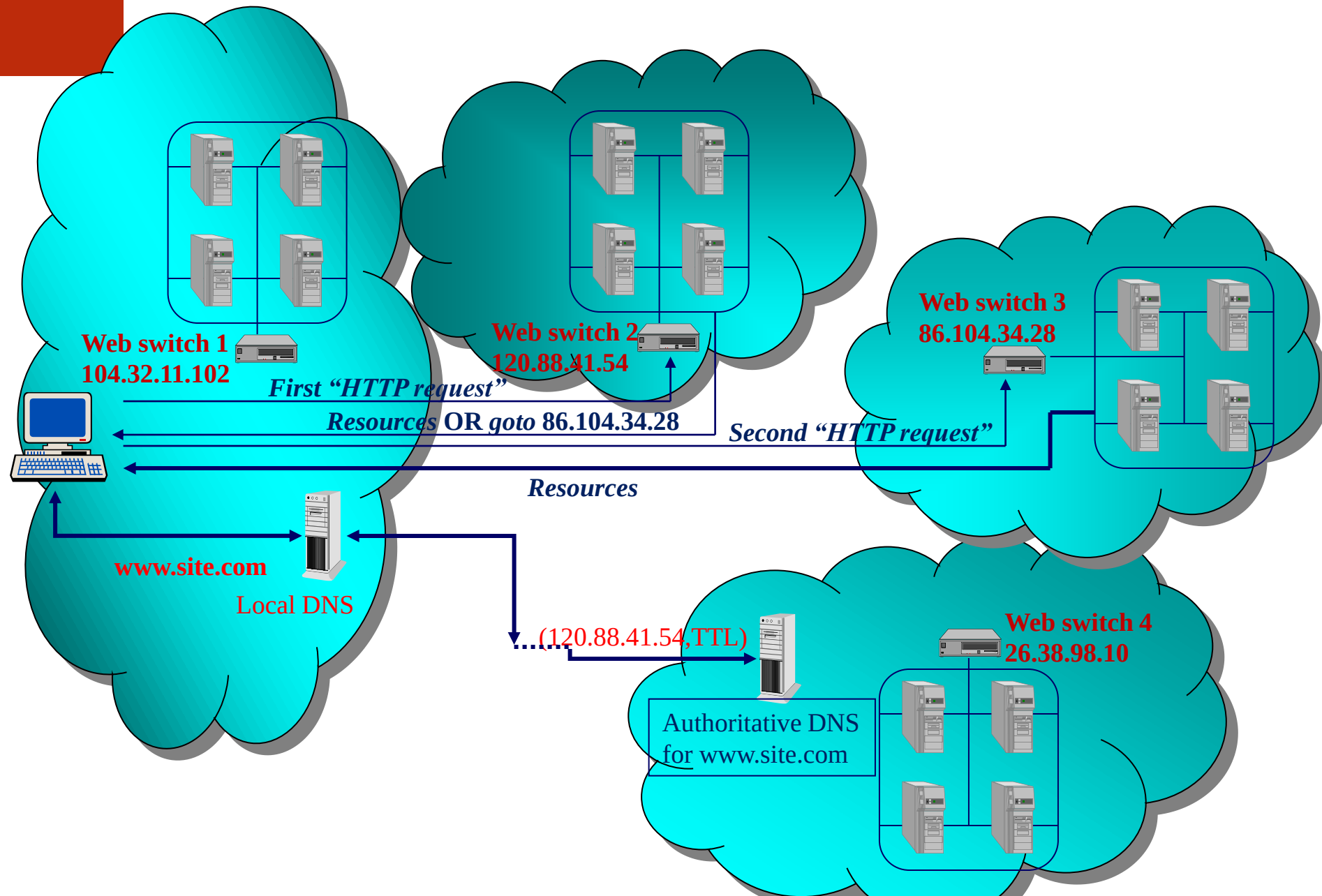
Is there correlation between hop counts and round-trip time?

- “Old” measures: close to zero [’90]
- “Recent” measures: strong, reasonably strong

WHY?



Geo-distributed system (3-level dispatching)



HTTP redirection mechanism

- The redirection mechanism is part of the HTTP protocol and is supported by current browser and server software
- DNS and Web switch use centralized scheduling disciplines
- HTTP redirection is a distributed scheduling policy, in which all Web clusters independently can participate in re-assigning requests
- Redirection is transparent to the user (**not to the client!**)



HTTP redirection

message header

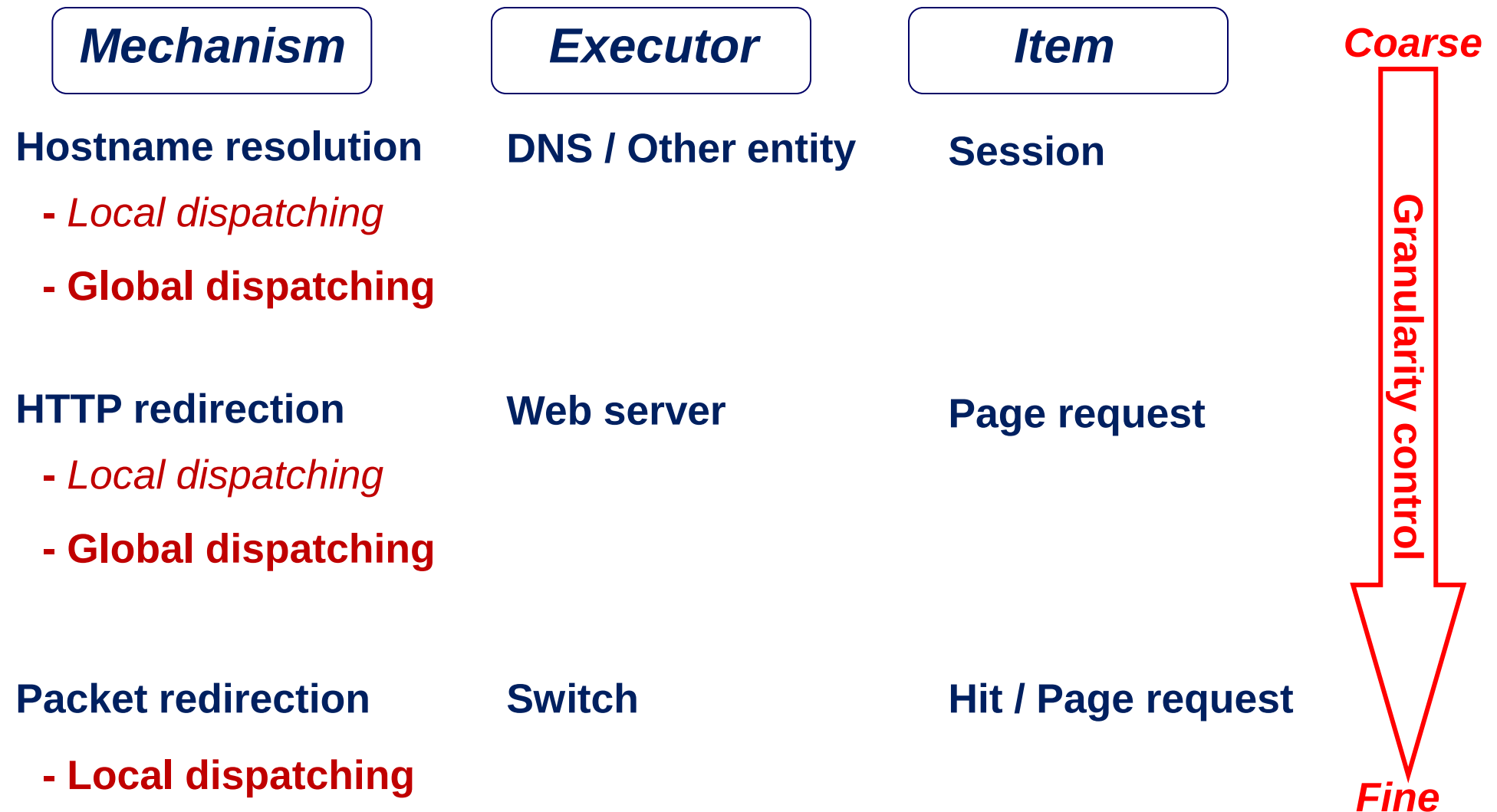
HTTP OK status code

302 - “Moved temporarily” to a new location

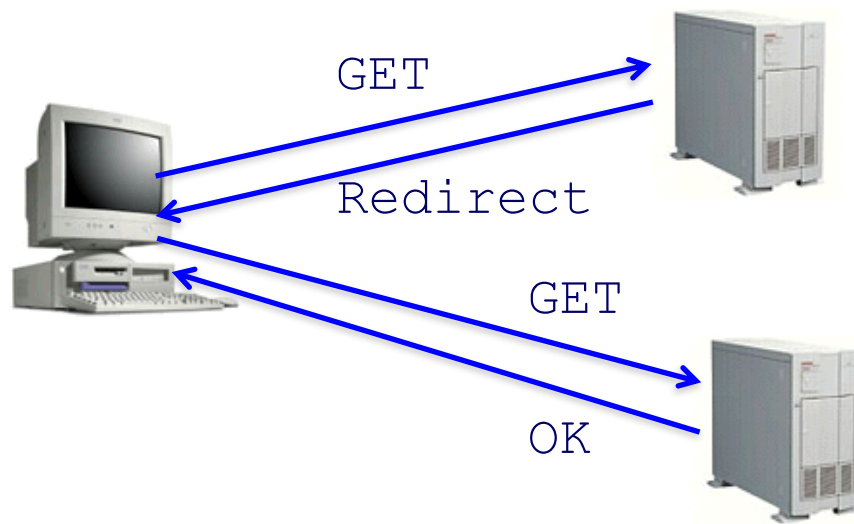
- “New location”
 - Redirection to an IP address (**better performance**)
 - Redirection to a hostname
- *HTTP redirection* is easy to implement **but works for HTTP services only**
- **As an alternative, *IP tunneling* can be used when it is necessary to provide other services**



Dispatching mechanisms: *summary*



Server selection mechanism: *HTTP redirection*



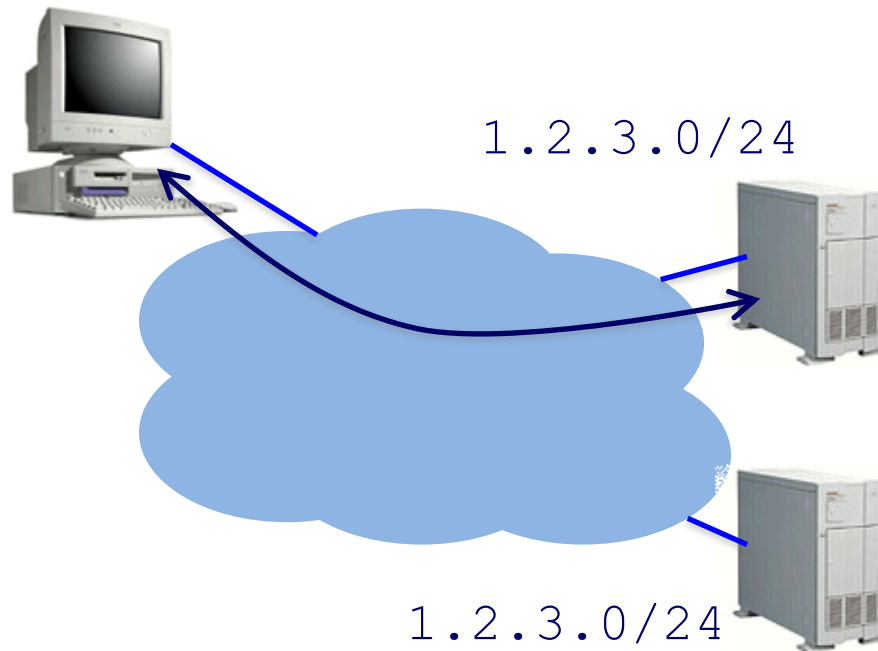
Advantages

- Fine-grain control
- Selection based on client IP address

Disadvantages

- Extra round-trips for TCP connection to server
- Overhead on the server

Server selection mechanism: *Anycast routing*



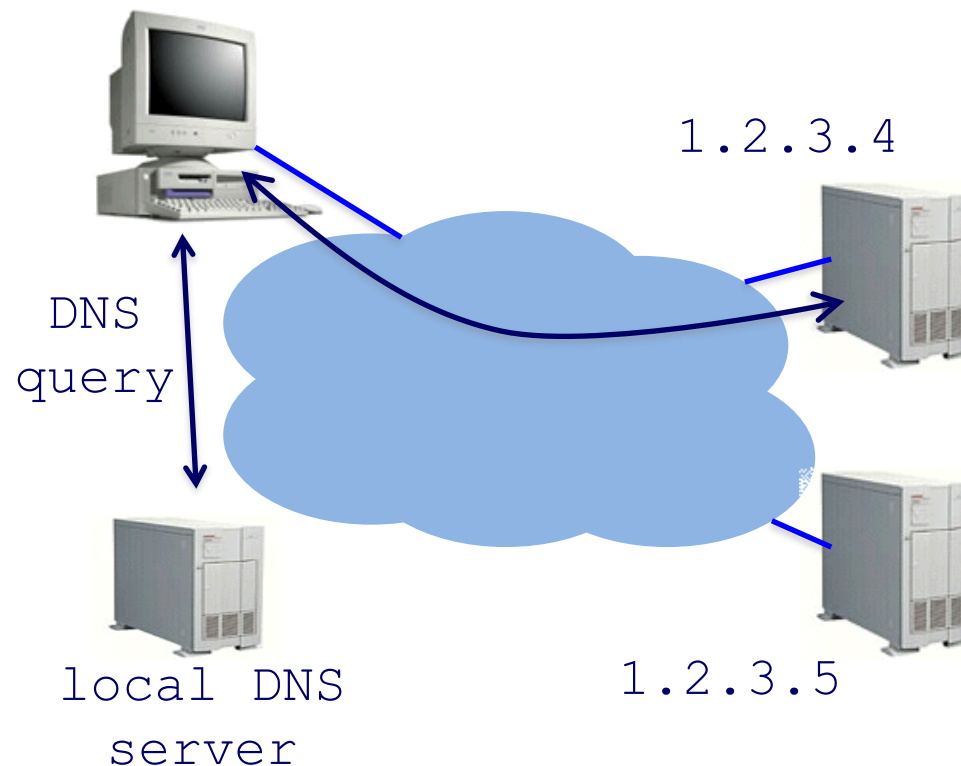
Advantages

- No extra round trips
- Route to nearby server

Disadvantages

- Does not consider server load
- Different packets may go to different servers
- Used only for simple request-response apps

Server selection mechanism: *DNS-based*



Advantages

- Avoid TCP set-up delay
- DNS caching reduces overhead
- Relatively fine control

Disadvantage

- Based on IP address of local DNS server
- DNS TTL limits adaptation with *hidden load* risks

Geographically distributed systems

- **Two-level dispatching**
 - Simplicity
 - High control on load reaching the Web cluster
 - Slow reaction to an overloaded Web cluster
- **Three-level dispatching**
 - Complex dispatching
 - Redirection valid only for HTTP-based services
 - Guarantees immediate actions to shift the load away from an overloaded Web cluster

This is a typical example where ***adaptive dispatching*** can work well → Use two-level dispatching when the system state is in good conditions; switch to three-level otherwise



Several alternatives

- Two-levels vs. Three-levels dispatching
- Which **selection** policy? *Redirect-All* vs. *Redirect-Heavy*
- Which **location** policy?
- Dispatching algorithms
 - Level 1 (*DNS*): e.g., proximity
 - Level 2 (*Switch*): e.g., Weighted Round Robin
 - Level 3 (*Web servers*)
 - Selection policy (“*which requests have to be redirected?*”): e.g.,
 - Redirect all requests
 - Redirect just “heavy” requests in terms of size or processing
 - Location policy (“*towards which system?*”): e.g.,
 - Round Robin, Least loaded system, System proximity, ...
 - Combinations: pick



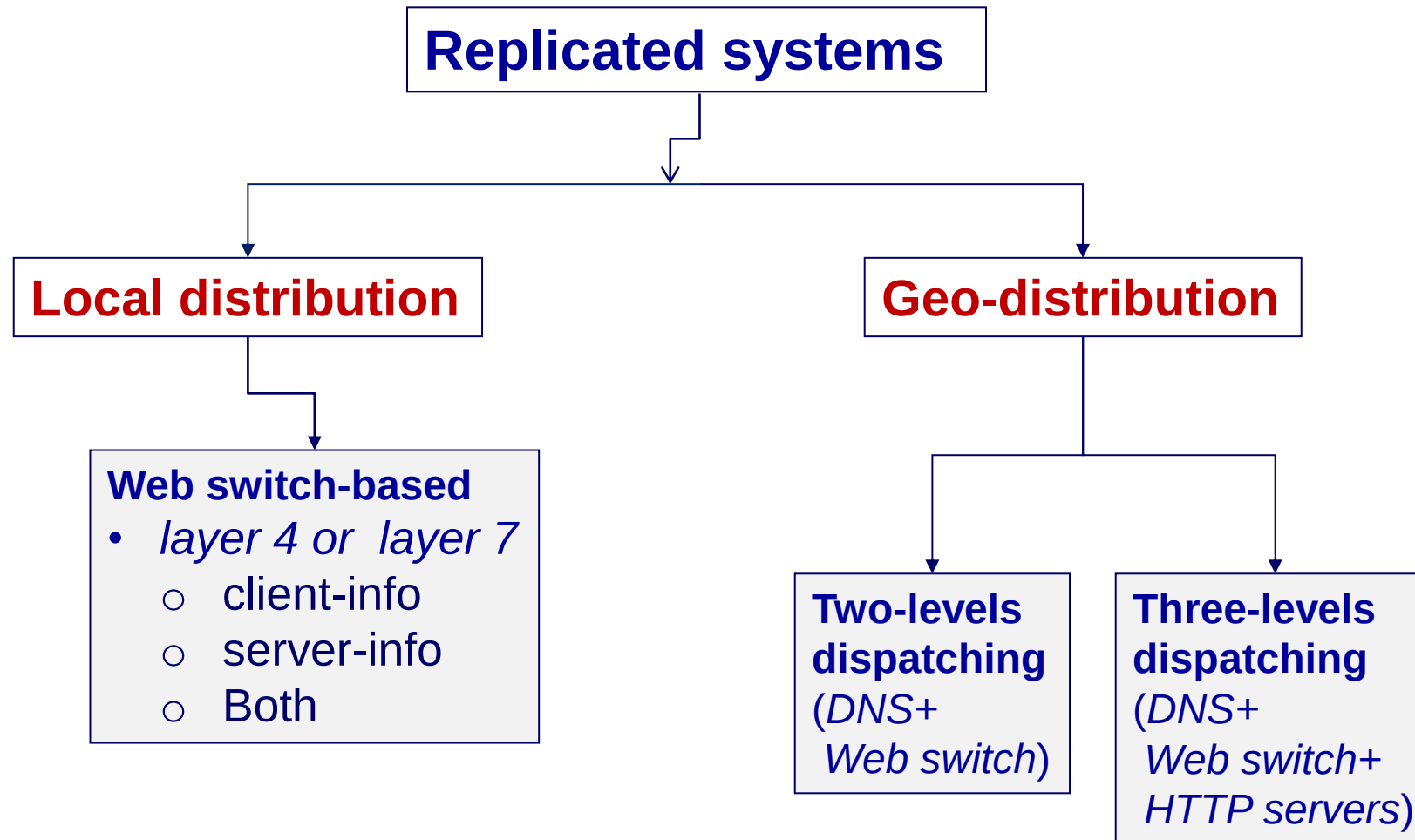
Server selection policies

- **Live server**
 - For availability
- **Low load (not always the lowest)**
 - To balance load across the servers
- **Closest**
 - Nearest geographically, latency, or measured in round-trip time
- **Best performance**
 - Throughput, ...
- **Others**
 - Cheapest bandwidth, electricity savings, ...

Requires continuous monitoring of liveness, load, and performance



Summary



Geo-replication of IDCs



If you are a cloud provider or you manage a popular service, one datacenter is insufficient

Multiple Internet Data Centers (and not just one) represent the back-end infrastructure used by cloud providers and largest digital companies to provide agile, elastic, reliable, secure services delivered through the Internet



Questions related to geo-replication

There are also challenges to address at the design level:

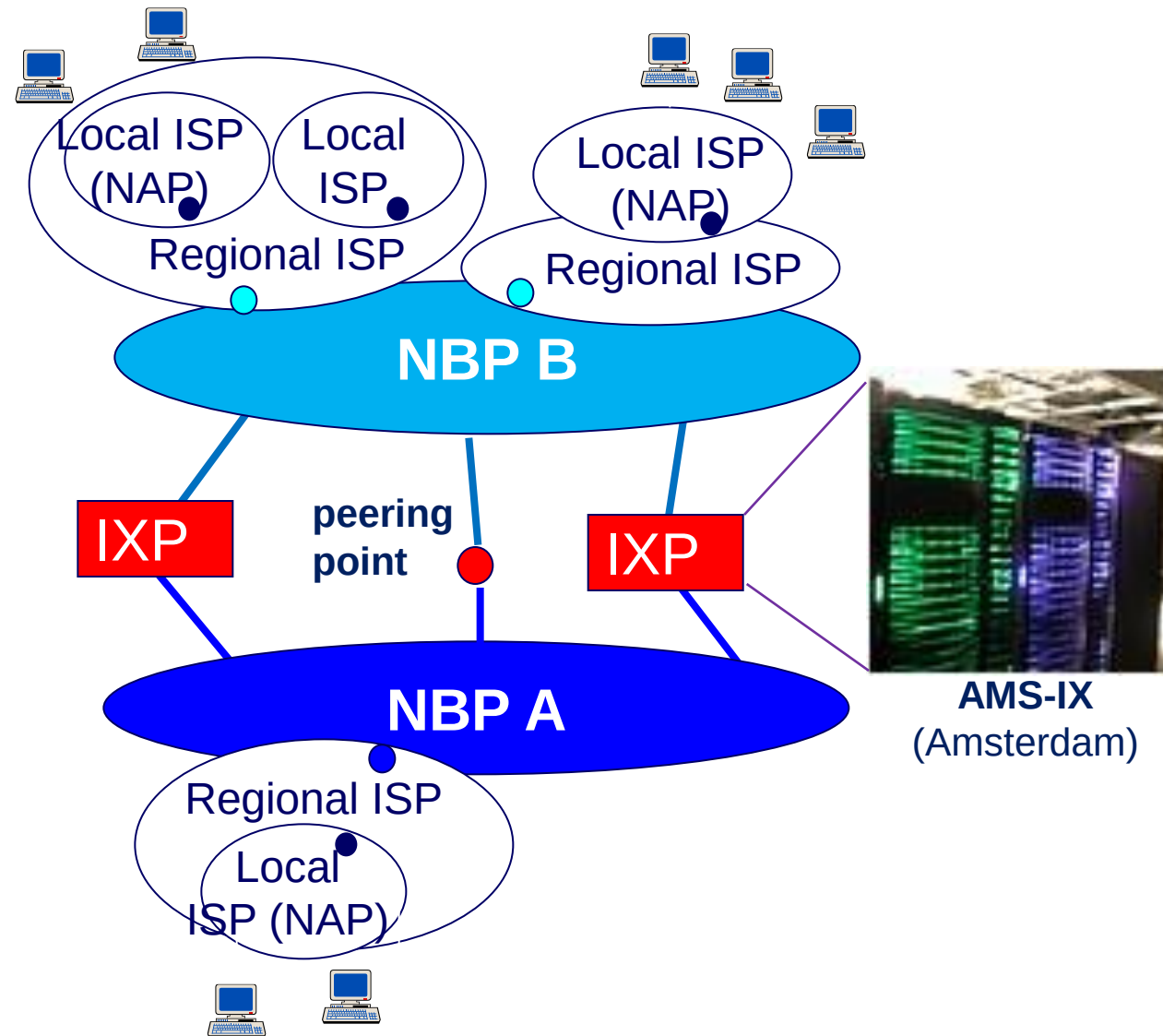
- **Where to place servers?** (*discussed*)
- **Where to place the primary node (IDC)?**
- **Where to place secondary nodes?**
- **How big to make each node?**

1. Few **mega-datacenters**?
2. Many regional datacenters (also called **mini or micro-datacenters**)?
3. Combinations of the two?

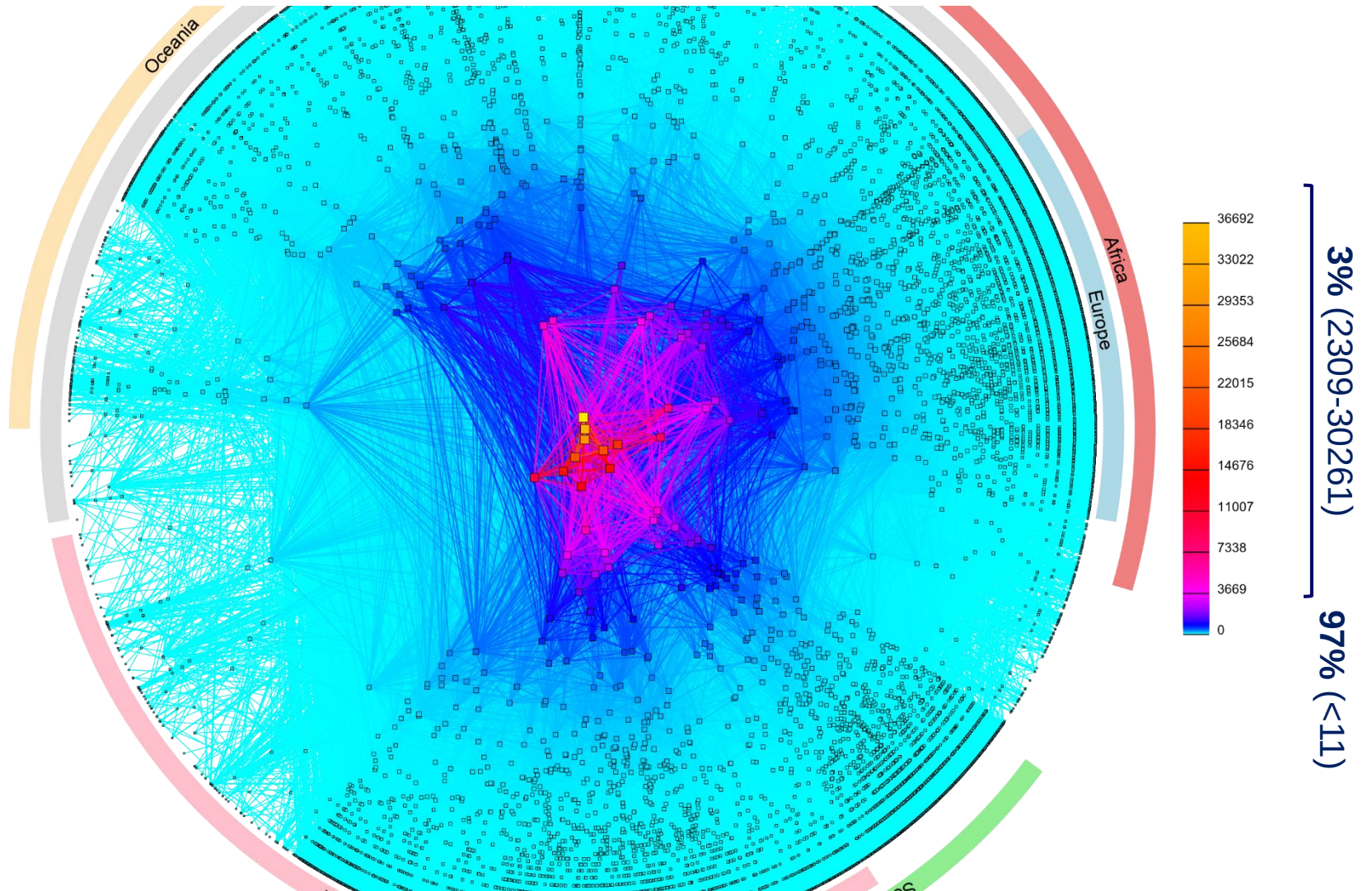


You have also to take into account how ASes are interconnected (<https://he.net/3d-map/>)

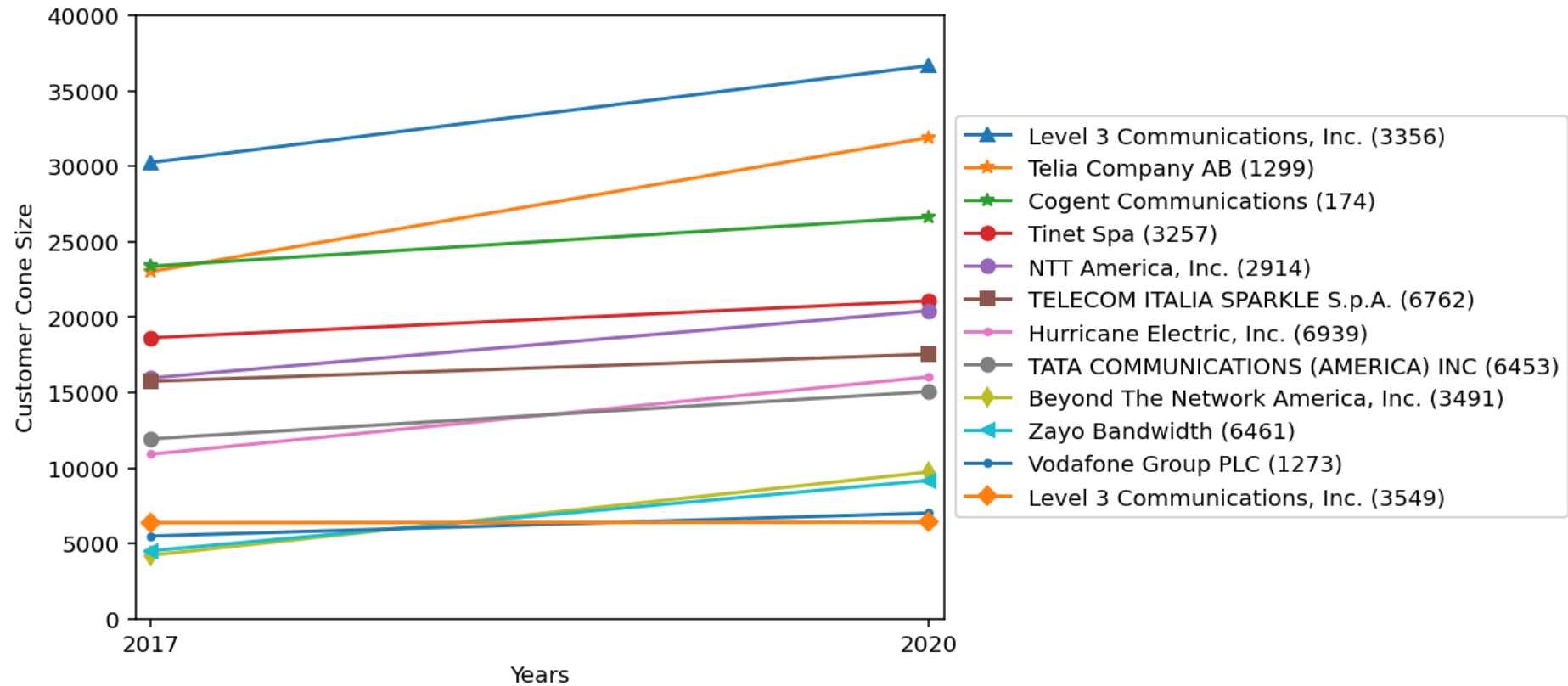
- **peering point**
(private)
- **Internet Exchange Point (IXP or IX)**



AS interconnections (CAIDA, 2020)



Top ten ASes



The importance of network connection

